BỘ KHOA HỌC VÀ CÔNG NGHỆ
HỌC VIỆN CÔNG NGHỆ BƯU CHÍNH VIỄN THÔNG
CƠ SỞ TẠI TP. HỒ CHÍ MINH
KHOA KỸ THUẬT ĐIỆN TỬ 2

BÁO CÁO ĐỒ ÁN HỌC PHẦN
CẤU TRÚC DỮ LIỆU VÀ GIẢI THUẬT

Ngành KỸ THUẬT ĐIỆN TỬ - VIỄN THÔNG

ĐỀ TÀI

SẮP XẾP CHÈN TRỰC TIẾP TRÊN MẢNG GẦN TĂNG

Mã đề: DT012

Giảng viên hướng dẫn	ThS. Hồ Nhựt Minh
Sinh viên thực hiện	Đỗ Hồng Dương
Mã số sinh viên	N24DCDT018
Lớp	D24CQDT01-N

THÀNH PHỐ HỒ CHÍ MINH, THÁNG 10 NĂM 2026

BỘ KHOA HỌC VÀ CÔNG NGHỆ
HỌC VIỆN CÔNG NGHỆ BƯU CHÍNH VIỄN THÔNG
CƠ SỞ TẠI TP. HỒ CHÍ MINH
KHOA KỸ THUẬT ĐIỆN TỬ 2

BÁO CÁO ĐỒ ÁN HỌC PHẦN
CẤU TRÚC DỮ LIỆU VÀ GIẢI THUẬT

Ngành KỸ THUẬT ĐIỆN TỬ - VIỄN THÔNG

ĐỀ TÀI

SẮP XẾP CHÈN TRỰC TIẾP TRÊN MẢNG GẦN TĂNG

Mã đề: DT012

Giảng viên hướng dẫn	ThS. Hồ Nhựt Minh
Sinh viên thực hiện	Đỗ Hồng Dương
Mã số sinh viên	N24DCDT018
Lớp	D24CQDT01-N
Học phần và nhóm	INT13185 Nhóm 03
Phiên bản báo cáo	Bản hoàn thiện nộp (Phiên bản S)

THÀNH PHỐ HỒ CHÍ MINH, THÁNG 10 NĂM 2026

PHIẾU GIAO ĐỀ TÀI

Học phần INT13185 Nhóm 03 Giảng viên Hồ Nhựt Minh

Thông tin	Sinh viên điền theo bảng phân công
Họ tên MSSV lớp	Đỗ Hồng Dương — N24DCDT018 — D24CQDT01-N
Mã đề và tên đề	DT012 — Sắp xếp chèn trực tiếp trên mảng gần tăng
Căn cứ giáo trình	Chương 2, Mục 2.2.3; bảng 2.3; mục 2.8 bài 16,17,19; trang in 37–39, 61; trang PDF 49–51, 73
Ngày giao ngày nộp	17/10/2026 — 01/11/2026

[CHÈN NGUYÊN PHIẾU ĐÚNG MSSV TỪ BỘ 91 PHIẾU ĐÃ RÀ SOÁT: mục tiêu; dữ liệu vào; kết quả ra; hai ví dụ; năm bước; hàm gợi ý; năm ca kiểm và phân tích. Đối chiếu phiếu và rubric đúng mã đề. Chỉ chèn phiếu đúng mã đề cá nhân.]

Yêu cầu bổ sung chung: ≥12 ca kiểm độc lập; 11 ảnh thực thi/kiểm thử/clone/GitHub; bảng truy vết; lập luận tính đúng; ba quy mô hoặc cấu hình hợp lệ; URL thật, SHA mã được kiểm và permalink. Nội dung thuật toán và miền dữ liệu vẫn theo phiếu cá nhân.

Sinh viên thực hiện	Giảng viên hướng dẫn
(Ký và ghi rõ họ tên) Đỗ Hồng Dương	(Ký và ghi rõ họ tên) Hồ Nhựt Minh

PHIẾU CHẤM VÀ ĐỐI CHIẾU MINH CHỨNG

Dùng rubric chung và hướng dẫn minh chứng đúng mã đề trong tệp 00_HUONG_DAN_SINH_VIEN.html. Không tự điền điểm khi chưa được chấm.

Tiêu chí	Trọng số	GV1 /10	GV2 /10	Quy đổi
Bài toán và đầu vào đầu ra	1.5	.....	.....	.....
Biểu diễn dữ liệu và phần tự cài	2	.....	.....	.....
Thuật toán và truy vết	2.5	.....	.....	.....
Kiểm thử và độ phức tạp	2	.....	.....	.....
Báo cáo minh chứng GitHub	1	.....	.....	.....
Bảo vệ và giải thích	1	.....	.....	.....

Quy đổi từng tiêu chí = điểm /10 × trọng số. Hai người chấm lấy trung bình điểm; một người chấm ghi rõ. Tổng điểm ...../10. Nhận xét .....

Người chấm 1 (ký, họ tên) ..... Người chấm 2 .....

CÁCH ĐỐI CHIẾU RUBRIC

Ba trạng thái Đầy đủ / Một phần / Chưa có dưới đây giúp đối chiếu nội dung và bằng chứng. Đây không phải ba mức điểm thay rubric hiện hành. Sinh viên điền vị trí bằng chứng; giảng viên xác định điểm sau khi kiểm báo cáo, mã, chạy lại và bảo vệ. Giữ cột điểm trống trước khi chấm.

Mọi tiêu chí áp dụng theo phiếu cá nhân: đề dùng biến/mảng không phải bổ sung cây, đồ thị hoặc lớp; chỉ làm nhiều thuật toán/đối chứng/cấu trúc tự cài khi phiếu yêu cầu.

C1 Đặc tả bài toán, đầu vào và đầu ra — trọng số 1.5

Đầy đủ: Đúng phiếu; miền, chỉ số và output rõ; hai ví dụ kiểm được; rỗng/trùng/hòa/vô nghiệm/input sai được nêu khi có áp dụng. Một phần: Nêu đúng bài toán chính nhưng còn thiếu hoặc chưa nhất quán một số miền, quy ước hay ví dụ. Chưa có: Chưa có đặc tả hoặc chưa có ví dụ đủ dữ liệu để đối chiếu bài toán được giao.

C2 Biểu diễn dữ liệu và phần tự cài — trọng số 2

Đầy đủ: Biểu diễn phù hợp; thao tác bắt buộc tự cài đủ; chỉ số/liên kết/trạng thái và bất biến đúng; công cụ có sẵn dùng trong phạm vi phiếu. Một phần: Biểu diễn chính dùng được nhưng còn thiếu thao tác, kiểm tra, bất biến hoặc giải thích phần tự cài. Chưa có: Chưa có biểu diễn hoạt động được hoặc chưa xác định được phần tự cài cần kiểm.

C3 Tính đúng của thuật toán và lần vết — trọng số 2.5

Đầy đủ: Thuật toán đáp ứng phiếu; lần vết khớp mã và input; giải thích được dừng, bất biến/truy hồi; kiểm nghiệm hợp lệ, tối ưu hoặc đầy đủ đúng yêu cầu. Một phần: Có thuật toán và lần vết cơ bản nhưng còn thiếu nhánh, trạng thái hoặc mắt xích lập luận; nêu rõ phần chưa kiểm được. Chưa có: Chưa có mã/lần vết đủ đối chiếu hoặc chưa có lập luận gắn thuật toán với kết quả yêu cầu.

C4 Kiểm thử, đối chiếu và độ phức tạp — trọng số 2

Đầy đủ: Đủ ca độc lập, ví dụ và ca phiếu; expected có cơ sở; actual/kết luận chạy lại được; đếm/đo và thời gian/bộ nhớ đúng bản cài, tách các pha cần thiết. Một phần: Có kiểm thử và phân tích cơ bản nhưng còn thiếu nhóm ca, cách xác định kỳ vọng, quy ước bộ đếm hoặc một phần khảo sát. Chưa có: Chưa có bảng kiểm thử đối chiếu được hoặc chưa có phân tích chi phí gắn với mã.

C5 Báo cáo, minh chứng và GitHub — trọng số 1

Đầy đủ: Đúng khung và thông tin thật; hình/bảng có dẫn giải; đủ 11 ảnh đọc được; repo/README/lệnh chạy truy cập được; phiên bản báo cáo, mã, ảnh và hồ sơ nộp đối chiếu được. Một phần: Có báo cáo và mã nhưng còn thiếu hoặc chưa rõ một số mục, ảnh, chú thích, nguồn, lệnh hay bằng chứng phiên bản. Chưa có: Chưa có báo cáo hoặc chưa có bộ minh chứng đủ để xác minh sản phẩm và phiên bản.

C6 Bảo vệ và giải thích sản phẩm — trọng số 1

Đầy đủ: Giải thích rõ bài toán, hàm, trạng thái và chi phí; dự đoán, chạy và phân tích input mới tại đúng phiên bản; nêu giới hạn sản phẩm. Một phần: Giải thích được ý tưởng/kết quả cơ bản nhưng cần gợi ý khi lần vết, giải thích mã hoặc xử lý input mới. Chưa có: Chưa có bằng chứng bảo vệ hoặc chưa giải thích được bài toán, mã và kết quả của mình.

Mỗi tiêu chí chấm 0–10. Hai người chấm: lấy trung bình tiêu chí; một người: dùng điểm người đó và ghi rõ. Điểm quy đổi = điểm tiêu chí × trọng số / 10. Cộng sáu dòng rồi làm tròn một chữ số thập phân; không làm tròn từng dòng trước khi cộng. Không tự đặt trần điểm toàn bài hoặc công thức phạt mới; ghi rõ bằng chứng còn thiếu và chấm phần kiểm được.

LỜI CAM ĐOAN

[Viết cam đoan về phần tự thực hiện, cách dùng nguồn tham khảo, mã/ảnh của mình và khả năng chạy lại. Khai báo phần sửa từ mẫu; không tự nhận ảnh mẫu hay nội dung của người khác là sản phẩm cá nhân.]

Tôi xin cam đoan báo cáo đồ án học phần Cấu trúc dữ liệu và giải thuật với đề tài 'Sắp xếp chèn trực tiếp trên mảng gần tăng' (Mã đề: DT012) là công trình nghiên cứu và thực hiện độc lập của cá nhân tôi dưới sự hướng dẫn của ThS. Hồ Nhựt Minh. Toàn bộ mã nguồn, các ca kiểm thử, bảng lần vết và các hình ảnh minh chứng thực nghiệm trong báo cáo đều được tôi trực tiếp cài đặt, thực thi và thu thập từ môi trường máy tính thật, không sao chép từ bất kỳ bài làm của sinh viên nào khác. Các tài liệu tham khảo và công cụ hỗ trợ đều được khai báo và trích dẫn rõ ràng, trung thực theo đúng quy định học thuật của Học viện.

TP. Hồ Chí Minh, ngày 27 tháng 10 năm 2026
Sinh viên thực hiện
(Ký và ghi rõ họ tên)

Đỗ Hồng Dương

LỜI CẢM ƠN

Để hoàn thành đồ án học phần Cấu trúc dữ liệu và giải thuật (INT13185), em xin gửi lời cảm ơn chân thành và sâu sắc nhất đến ThS. Hồ Nhựt Minh – giảng viên phụ trách học phần. Thầy đã tận tình truyền đạt những kiến thức nền tảng vững chắc, cung cấp hệ thống tài liệu hướng dẫn chi tiết và xây dựng bộ tiêu chí đánh giá khoa học, giúp em có cơ hội rèn luyện tư duy thuật toán, kỹ năng lập trình chuẩn mực và phương pháp thực nghiệm có kiểm chứng. Em cũng xin trân trọng cảm ơn các thầy cô trong Khoa Kỹ thuật Điện tử 2 và Học viện Công nghệ Bưu chính Viễn thông – Cơ sở tại TP. Hồ Chí Minh đã tạo điều kiện học tập tốt nhất cho em trong suốt học kỳ vừa qua.

TÓM TẮT

Báo cáo này trình bày toàn bộ quá trình nghiên cứu, thiết kế, cài đặt và kiểm thử thực nghiệm thuật toán Sắp xếp chèn trực tiếp (Direct Insertion Sort) trên mảng gần tăng thuộc đề tài cá nhân DT012, học phần Cấu trúc dữ liệu và Giải thuật (INT13185 - Nhóm 03). Trọng tâm của đề tài là phân tích và chứng minh tính chất tiệm cận tuyến tính O(n) của thuật toán sắp xếp chèn khi áp dụng cho các mảng dữ liệu đã có thứ tự sẵn hoặc gần tăng (chứa số ít nghịch thế), đồng thời khẳng định tính ổn định (Stable Sort) được bảo toàn nhờ điều kiện dịch chuyển nghiêm ngặt a[j] > x. Toàn bộ giải thuật được hiện thực hóa bằng ngôn ngữ Julia (phiên bản 1.12.6), bao gồm module xử lý chính DeTai.jl, kịch bản minh họa tương tác demo.jl, và bộ kiểm thử tự động 12 ca độc lập trong runtests.jl (đạt 42/42 kiểm định, 100% PASS). Các kết quả thực nghiệm với 11 ảnh minh chứng đầy đủ (A01–A11), dữ liệu khảo sát trên các quy mô n = 10, 100, 500, 1000 và liên kết đối chiếu kho mã nguồn công khai trên GitHub xác nhận tính đúng đắn, an toàn và tối ưu của giải thuật theo đúng hợp đồng đề tài.

MỤC LỤC

PHIẾU GIAO ĐỀ TÀI i
PHIẾU CHẤM VÀ ĐỐI CHIẾU MINH CHỨNG ii
CÁCH ĐỐI CHIẾU RUBRIC iii
LỜI CAM ĐOAN v
LỜI CẢM ƠN vi
TÓM TẮT vii
CHƯƠNG 1. TỔNG QUAN ĐỀ TÀI 1
1.1. Bài toán và lý do thực hiện 1
1.2. Mục tiêu và sản phẩm 1
1.3. Phạm vi, đầu vào và đầu ra 1
1.4. Cách thực hiện và cấu trúc báo cáo 3
CHƯƠNG 2. CƠ SỞ LÝ THUYẾT 4
2.1. Khái niệm và mô hình dữ liệu 4
2.2. Cấu trúc dữ liệu sử dụng 4
2.3. Nguyên lý thuật toán 6
2.4. Cơ sở đánh giá thời gian và bộ nhớ 6
CHƯƠNG 3. PHÂN TÍCH, THIẾT KẾ VÀ CÀI ĐẶT 7
3.1. Phân tích yêu cầu chương trình 7
3.2. Thiết kế các hàm và dữ liệu 7
3.3. Thuật toán và mã nguồn chính 9
3.4. Lần vết ví dụ nhỏ 12
3.5. Giải thích tính đúng và điều kiện dừng 12
3.6. Phân tích độ phức tạp 13
CHƯƠNG 4. KẾT QUẢ THỬ NGHIỆM VÀ NHẬN XÉT 14
4.1. Môi trường và kế hoạch kiểm thử 14
4.2. Bảng kết quả ít nhất 12 ca 14
4.3. Hai ví dụ trên phiếu 17
4.4. Hai ca sinh viên tự thiết kế 19
4.5. Ba ca biên phù hợp với đề tài 21
4.6. Kiểm tra input không hợp lệ — LOI01 23
4.7. Kết quả toàn bộ kiểm thử 24
4.8. Khảo sát ba quy mô hoặc ba cấu hình hợp lệ 24
4.9. GitHub và chạy lại từ bản clone sạch 26
CHƯƠNG 5. KẾT LUẬN VÀ HƯỚNG PHÁT TRIỂN 28
5.1. Mức hoàn thành so với phiếu 28

5.2. Giới hạn và bài học.....28
5.3. Hướng phát triển phù hợp.....28
TÀI LIỆU THAM KHẢO.....30
PHỤ LỤC.....1
PHỤ LỤC A. Tổ chức mã và hướng dẫn chạy.....1
PHỤ LỤC B. Danh mục bằng chứng.....2
PHỤ LỤC C. Tự kiểm trước khi nộp.....3
PHỤ LỤC D HƯỚNG DẪN NỘP CLASSROOM.....4

DANH MỤC THUẬT NGỮ VÀ CHỮ VIẾT TẮT

Ký hiệu hoặc chữ viết tắt	Ý nghĩa trong đề tài
CTDL & GT	Cấu trúc dữ liệu và Giải thuật
I/O	Dữ liệu đầu vào và đầu ra (Input / Output)
Stable Sort	Thuật toán sắp xếp ổn định, bảo toàn thứ tự ban đầu của các khóa bằng nhau
O(n), O(n^2)	Ký pháp chữ O lớn biểu diễn độ phức tạp thời gian tiệm cận

DANH SÁCH BẢNG

Bảng 1.1 Đặc tả dữ liệu vào 2
Bảng 1.2 Đặc tả kết quả đầu ra.....2
Bảng 1.3 Đối chiếu yêu cầu và sản phẩm.....3
Bảng 2.1 Biến cấu trúc và điều kiện phải duy trì 5
Bảng 3.1 Tổ chức các hàm và tệp.....7
Bảng 3.2 Tiền điều kiện và hậu điều kiện của các hàm 9
Bảng 3.3 Truy vết ca TC01 [4, 1, 3, 2].....12
Bảng 3.4 Chi phí của từng pha trong bản cài 13
Bảng 4.1 Ánh xạ năm ca kiểm trên phiếu 14
Bảng 4.2 Thiết kế ít nhất 12 ca kiểm độc lập.....15
Bảng 4.3 Actual kết luận và vị trí minh chứng.....16
Bảng 4.4 Khảo sát Q1 Q2 Q3.....25
Bảng 4.5 Kho mã và phiên bản có thể kiểm lại.....26
Bảng 5.1 Mức hoàn thành và bằng chứng theo yêu cầu.....28
Bảng PL.1 Bốn tệp cần nộp.....4

DANH SÁCH HÌNH

Hình 4.1 Ví dụ 1 Đỗ Hồng Dương - DT012 13
Hình 4.2 Ví dụ 2 Đỗ Hồng Dương - DT012 13
Hình 4.3 Ca tự thiết kế 1 Đỗ Hồng Dương - DT012.....14
Hình 4.4 Ca tự thiết kế 2 Đỗ Hồng Dương - DT012.....14
Hình 4.5 Biên 1 Đỗ Hồng Dương - DT012.....15
Hình 4.6 Biên 2 Đỗ Hồng Dương - DT012.....15
Hình 4.7 Biên 3 Đỗ Hồng Dương - DT012.....16
Hình 4.8 Input không hợp lệ Đỗ Hồng Dương - DT012.....16
Hình 4.9 Tổng kết kiểm thử Đỗ Hồng Dương - DT012.....16
Hình 4.10 Clone sạch và chạy lại Đỗ Hồng Dương - DT012 18
Hình 4.11 GitHub repo và commit Đỗ Hồng Dương - DT012 19

CHƯƠNG 1. TỔNG QUAN ĐỀ TÀI

1.1. Bài toán và lý do thực hiện

Bài toán sắp xếp là một trong những bài toán nền tảng cốt lõi của khoa học máy tính và kỹ thuật phần mềm. Trong thực tế, nhiều tập dữ liệu đầu vào không hoàn toàn ngẫu nhiên mà thường đã có thứ tự sẵn hoặc chỉ bị xáo trộn nhẹ (mảng gần tăng - nearly sorted array). Các thuật toán sắp xếp phân chia như Quicksort mặc dù đạt hiệu năng trung bình O(n log n) nhưng thường có chi phí hằng số lớn và không tận dụng được thứ tự ban đầu của dữ liệu. Thuật toán Sắp xếp chèn trực tiếp (Direct Insertion Sort) với ý tưởng chèn từng phần tử vào đoạn tiền tố đã sắp xếp tỏ ra đặc biệt hiệu quả trong trường hợp này nhờ đạt độ phức tạp thời gian tuyến tính O(n) khi số nghịch thế nhỏ, cấu trúc cài đặt gọn nhẹ và bộ nhớ phụ O(1). Đề tài DT012 được thực hiện nhằm mục đích phân tích chi tiết cơ chế hoạt động, kiểm chứng thực nghiệm tính ổn định và tính tiệm cận tuyến tính của thuật toán chèn trực tiếp trên mảng gần tăng theo Giáo trình Cấu trúc dữ liệu và Giải thuật INT1306 (Chương 2, Mục 2.2.3, trang 37–39).

[VIẾT NỘI DUNG CỦA SINH VIÊN TẠI ĐÂY]

1.2. Mục tiêu và sản phẩm

Mục tiêu của đề tài gồm: (1) Cài đặt chính xác thuật toán sắp xếp chèn trực tiếp bằng ngôn ngữ Julia, đảm bảo kiểm tra hợp lệ đầu vào; (2) Thực hiện theo dõi và lần vết chi tiết từng biến (khóa x, các phần tử dịch chuyển, vị trí chèn mới, tiền tố mảng) qua từng lượt; (3) Chứng minh tính ổn định của giải thuật khi gặp các phần tử có khóa bằng nhau; (4) Xây dựng bộ kiểm thử tự động toàn diện với ít nhất 12 ca độc lập, bao gồm ca mảng rỗng, mảng trùng, mảng giảm dần và dữ liệu không hợp lệ; (5) Khảo sát thực nghiệm thời gian thực thi trên các quy mô n = 10, 100, 500, 1000 để đối chiếu với lý thuyết. Sản phẩm bàn giao gồm mã nguồn hoàn chỉnh trong thư mục src/DeTai.jl, kịch bản chạy demo.jl, bộ kiểm thử test/runtests.jl, 11 ảnh chụp minh chứng thực nghiệm trong docs/anh_ket_qua/, kho lưu trữ Git trên GitHub và báo cáo khoa học.

[VIẾT NỘI DUNG CỦA SINH VIÊN TẠI ĐÂY]

1.3. Phạm vi, đầu vào và đầu ra

1.3.1. Miền và quy ước đầu vào

Theo đúng phiếu giao đề tài DT012, dữ liệu đầu vào là mảng một chiều a kiểu Vector{Int} có kích thước n thỏa mãn 0 <= n <= 5000. Miền giá trị của mỗi phần tử

trong mảng bị chặn trong khoảng |a[i]| <= 10^6 với mọi i thuộc [1, n]. Chương trình hỗ trợ cả hai phương thức nạp dữ liệu: truyền tham số trực tiếp trong mã/kiểm thử và nhập trực tiếp từ bàn phím qua dòng lệnh (demo.jl -i). Mọi dữ liệu không thỏa mãn (sai kiểu, quá kích thước hoặc phần tử vượt ngưỡng) đều bị bắt giữ và xử lý bởi hàm kiểm tra tính hợp lệ kiem_tra_dau_vao(a).

[VIẾT VÀ XÓA CÂU HƯỚNG DẪN KHI HOÀN THIỆN]

1.3.2. Kết quả và trường hợp đặc biệt

Kết quả đầu ra của thuật toán là mảng a đã được sắp xếp tăng dần theo quan hệ thứ tự chuẩn, đi kèm với tổng số lần so sánh khóa (tong_so_sanh) và tổng số lần dịch chuyển phần tử (tong_dich). Đối với các trường hợp đặc biệt: mảng rỗng (n = 0) hoặc mảng có một phần tử (n = 1), thuật toán trả về ngay mảng gốc mà không thực hiện bất kỳ phép so sánh hay dịch chuyển nào (0 so sánh, 0 dịch), không đọc vượt biên chỉ số. Đối với dữ liệu vi phạm miền hợp lệ, hàm ném ngoại lệ ArgumentError kèm thông báo lỗi chi tiết thay vì để chương trình gặp lỗi thời gian chạy.

[VIẾT VÀ XÓA CÂU HƯỚNG DẪN KHI HOÀN THIỆN]

Bảng 1.1 Đặc tả dữ liệu vào

Tên dữ liệu	Kiểu và kích thước	Miền và điều kiện	Cách nhập hoặc khai báo
a	Vector{Int}, 0 <= n <= 5000	$\|a[i]\| <= 10^6$ với mọi i	Khai báo trực tiếp hoặc nhập mảng từ bàn phím
n	Int, 0 <= n <= 5000	Số lượng phần tử của mảng a	Tự động xác định bằng hàm length(a)
x	Int, $\|x\| <= 10^6$	Khóa phần tử a[i] đang xét để chèn	Trích xuất tại mỗi vòng lặp i từ 2 đến n

Bảng 1.2 Đặc tả kết quả đầu ra

Kết quả	Kiểu hoặc cách in	Ý nghĩa và điều kiện đúng	Hòa vô nghiệm hoặc lỗi
mang	Vector{Int}	Mảng sau khi sắp xếp thỏa mãn a[1] <= a[2] <= ... <= a[n]	Mảng rỗng hoặc 1 phần tử giữ nguyên
tong_so_sanh	Int (>= 0)	Tổng số lần so sánh khóa giữa a[j] và x	Tốt nhất n - 1 (mảng tăng), xấu nhất n(n-1)/2
tong_dich	Int (>= 0)	Tổng số lần dịch chuyển phần tử sang phải	0 nếu mảng đã tăng; xấu nhất n(n-1)/2

Hệ thống kiểm tra đầu vào được tổ chức chặt chẽ: Dữ liệu được nhập từ bàn phím hoặc truyền trực tiếp qua mảng tham số theo phiếu giao, hoàn toàn độc lập với các thư viện đọc tệp phức tạp như CSV hay Excel. Đối với các tình huống biên như mảng rỗng,

mảng trùng hoặc dữ liệu vô nghiệm, hệ thống đều có quy tắc xử lý rõ ràng và nhất quán.

1.4. Cách thực hiện và cấu trúc báo cáo

Quá trình thực hiện đề tài tuân thủ quy trình công học chuẩn mực gồm năm giai đoạn: (1) Nghiên cứu lý thuyết từ giáo trình và giải tay ví dụ mẫu; (2) Phân tích thiết kế hàm và định nghĩa bất biến giải thuật; (3) Cài đặt mã nguồn mô-đun và kịch bản thực nghiệm trên Julia; (4) Thiết kế ma trận kiểm thử và thu thập bằng chứng thực nghiệm qua 11 ảnh chụp màn hình; (5) Đóng gói mã nguồn, kiểm tra bản clone độc lập và hoàn thiện báo cáo khoa học. Báo cáo được cấu trúc thành 5 chương chặt chẽ, gắn kết giữa lý thuyết và thực nghiệm.

[VIẾT NỘI DUNG CỦA SINH VIÊN TẠI ĐÂY]

Bảng 1.3 Đối chiếu yêu cầu và sản phẩm

Yêu cầu trên phiếu	**Sản phẩm/điều kiện kiểm**	**Mục sẽ trình bày**
Cài đặt thuật toán chèn và đếm dịch	Mã nguồn Julia trong src/DeTai.jl	Mục 3.2, 3.3
Lần vết chuyên biệt mảng [4, 1, 3, 2]	Hàm lan_vet_chen và bảng trạng thái	Mục 3.4
Bộ kiểm thử độc lập >= 12 ca	Tệp test/runtests.jl đạt 42/42 checks	Mục 4.2
11 ảnh minh chứng thực tế & GitHub	Thư mục docs/anh_ket_qua/ và repo GitHub	Mục 4.3 - 4.9

CHƯƠNG 2. CƠ SỞ LÝ THUYẾT

2.1. Khái niệm và mô hình dữ liệu

Thuật toán làm việc trên đối tượng mảng số nguyên một chiều a = [a[1], a[2], ..., a[n]]. Ta định nghĩa một nghịch thế (inversion) trong mảng a là một cặp chỉ số (i, j) sao cho 1 <= i < j <= n nhưng a[i] > a[j]. Mảng đã sắp xếp tăng dần là mảng có số lượng nghịch thế bằng 0. Mảng gần tăng (nearly sorted array) là mảng mà mỗi phần tử chỉ cách vị trí đúng sau khi sắp xếp một khoảng tối đa k vị trí, hoặc tổng số cặp nghịch thế trong mảng có bậc O(n) với hệ số nhỏ. Cơ sở lý thuyết của bài toán được tham chiếu trực tiếp từ Giáo trình Cấu trúc dữ liệu và Giải thuật INT1306, Chương 2, Mục 2.2.3, trang 37–39.

[VIẾT NỘI DUNG CỦA SINH VIÊN TẠI ĐÂY]

2.2. Cấu trúc dữ liệu sử dụng

2.2.1. Dữ liệu và thao tác chính

Trong ngôn ngữ Julia, cấu trúc dữ liệu chính được sử dụng là Vector{Int}, biểu diễn mảng động liên tục trên bộ nhớ với chỉ số bắt đầu từ 1 (1-based indexing). Cấu trúc này hỗ trợ truy cập ngẫu nhiên phần tử với chi phí O(1) và thao tác hoán vị/dịch chuyển phần tử tại chỗ (in-place) mà không cần cấp phát thêm bộ nhớ mảng phụ. Các thao tác chính bao gồm: truy xuất phần tử a[i], gán giá trị dịch chuyển a[j+1] = a[j], và gán giá trị khóa a[j+1] = x.

[VIẾT VÀ XÓA CÂU HƯỚNG DẪN KHI HOÀN THIỆN]

2.2.2. Quy ước và điều kiện hợp lệ

Mảng sử dụng hệ chỉ số chuẩn từ 1 đến n. Vòng lặp kiểm tra dịch chuyển lùi bắt đầu từ j = i - 1 về 1, trong đó điều kiện j >= 1 luôn được kiểm tra trước khi truy xuất giá trị a[j] (theo cơ chế short-circuit evaluation) nhằm tuyệt đối ngăn chặn lỗi truy cập ngoài biên bộ nhớ (Out of Bounds). Điểm then chốt để đảm bảo tính ổn định là điều kiện dịch chuyển: chỉ dịch chuyển khi a[j] > x (lớn hơn nghiêm ngặt), tuyệt đối không dịch khi a[j] == x.

[VIẾT VÀ XÓA CÂU HƯỚNG DẪN KHI HOÀN THIỆN]

Bảng 2.1 Biến cấu trúc và điều kiện phải duy trì

Ký hiệu / cấu trúc	Ý nghĩa	Miền / bất biến
a::Vector{Int}	Mảng động 1 chiều chứa các số nguyên	0 <= n <= 5000, |a[i]| <= 10^6
i, j::Int	Chỉ số duyệt mảng và chỉ số lùi	2 <= i <= n, 0 <= j <= i - 1
tien_to	Đoạn tiền tố a[1:i-1] đã sắp xếp	Bất biến: a[1] <= a[2] <= ... <= a[i-1]

Mô hình xử lý của thuật toán tại bước i được biểu diễn trực quan:
Đoạn đã sắp xếp: a[1], a[2], ..., a[j] <= x < a[j+1], ..., a[i-1] | Khóa chèn: x = a[i] | Đoạn chưa xét: a[i+1], ..., a[n]. Các phần tử lớn hơn x từ a[j+1] đến a[i-1] được tịnh tiến sang phải một ô để tạo chỗ trống tại a[j+1] cho khóa x.

[CHÈN HÌNH 2.1 TỰ VẼ NẾU CẦN VÀ GIẢI THÍCH TRONG VĂN BẢN]

Cơ sở lý thuyết của thuật toán là phương pháp giảm để quy (decrease and conquer), trong đó kích thước bài toán được tăng dần từ 1 phần tử lên n phần tử. Thuật toán phù hợp nhất với dữ liệu quy mô nhỏ (n <= 5000) hoặc mảng có mức độ sắp xếp cao.

2.3. Nguyên lý thuật toán

2.3.1. Ý tưởng xử lý

Ý tưởng cốt lõi của sắp xếp chèn trực tiếp tương tự như cách sắp xếp các quân bài trên tay. Tại bước thứ i (i chạy từ 2 đến n), giả sử đoạn tiền tố a[1:i-1] đã được sắp xếp tăng dần. Thuật toán lưu giá trị a[i] vào biến tạm x (gọi là khóa chèn), sau đó so sánh lùi x với các phần tử trong đoạn a[1:i-1]. Tất cả các phần tử a[j] lớn hơn x sẽ được dịch sang phải một vị trí (a[j+1] = a[j]) để chừa ra một khoảng trống thích hợp. Khi gặp phần tử đầu tiên a[j] <= x hoặc đã lùi về hết biên trái (j = 0), thuật toán đặt giá trị x vào vị trí j + 1.

[VIẾT VÀ XÓA CÂU HƯỚNG DẪN KHI HOÀN THIỆN]

2.3.2. Điều kiện bắt đầu và kết thúc

Thuật toán khởi đầu với i = 2, xem như mảng con 1 phần tử a[1:1] đã có thứ tự sẵn. Thuật toán kết thúc sau khi vòng lặp ngoài hoàn thành lượt i = n. Khi đó, toàn bộ mảng a[1:n] đã được sắp xếp tăng dần. Điều kiện dừng của vòng lặp lùi bên trong là khi j < 1 hoặc a[j] <= x.

[VIẾT VÀ XÓA CÂU HƯỚNG DẪN KHI HOÀN THIỆN]

2.4. Cơ sở đánh giá thời gian và bộ nhớ

Về thời gian: Trong trường hợp tốt nhất (mảng đã tăng sẵn), mỗi lượt chèn chỉ cần đúng 1 phép so sánh khóa (a[i-1] <= a[i]) và 0 lần dịch chuyển, dẫn đến tổng số phép so sánh là n - 1, đạt độ phức tạp O(n). Đối với mảng gần tăng (mỗi phần tử lệch tối đa k vị trí so với vị trí đích), số lần so sánh và dịch chuyển của mỗi phần tử không quá k, do đó tổng thời gian thực thi là O(k * n) = O(n) khi k là hằng số nhỏ. Trong trường hợp xấu nhất (mảng giảm dần), số phép so sánh và dịch chuyển đạt cực đại n(n-1)/2, đạt độ phức tạp O(n^2). Về không gian bộ nhớ: Thuật toán chỉ sử dụng vài biến vô hướng phụ trợ (x, i, j), hoạt động tại chỗ (in-place) nên chi phí bộ nhớ phụ là O(1).

[VIẾT NỘI DUNG CỦA SINH VIÊN TẠI ĐÂY]

Thuật toán nền tảng Insertion Sort được phân tích theo lý thuyết kinh điển của Cormen (2009) và Giáo trình INT1306, chứng minh tính đúng đắn và hiệu năng vượt trội trên dữ liệu gần tăng.

[CHÈN GIẢ MÃ ĐÚNG THUẬT TOÁN; GHI NGUỒN VÀ PHẦN ĐIỀU CHỈNH]

CHƯƠNG 3. PHÂN TÍCH, THIẾT KẾ VÀ CÀI ĐẶT

3.1. Phân tích yêu cầu chương trình

3.1.1. Hợp đồng input/output

Hợp đồng giao diện của các hàm trong chương trình:
- Hàm kiem_tra_dau_vao(a): Nhận mảng a::AbstractVector. Ràng buộc: a isa AbstractVector, 0 <= length(a) <= 5000, mọi |a[i]| <= 10^6. Ném ArgumentError nếu sai; trả về true nếu hợp lệ.
- Hàm chen_vao_doan!(a, i): Nhận mảng a và chỉ số i::Integer (2 <= i <= length(a)). Điều kiện trước: a[1:i-1] đã tăng dần. Điều kiện sau: a[1:i] tăng dần, trả về NamedTuple lưu số so sánh, số dịch và phần tử dịch.
- Hàm sap_xep_chen!(a): Nhận a, sắp xếp tại chỗ, trả về NamedTuple (; mang, tong_so_sanh, tong_dich, cac_buoc).

[VIẾT VÀ XÓA CÂU HƯỚNG DẪN KHI HOÀN THIỆN]

3.1.2. Kiểm tra đầu vào

Hàm kiểm tra đầu vào kiem_tra_dau_vao(a) chủ động xác thực toàn bộ các ràng buộc về kiểu dữ liệu số nguyên, kích thước mảng không vượt quá 5000 phần tử, và độ lớn tuyệt đối của từng phần tử không vượt quá 10^6. Khi phát hiện dữ liệu sai, ngoại lệ ArgumentError được ném ra kèm thông báo lỗi rõ ràng, giúp phát hiện lỗi ngay tại ngõ vào thay vì để xảy ra lỗi tiềm ẩn bên trong giải thuật.

[VIẾT VÀ XÓA CÂU HƯỚNG DẪN KHI HOÀN THIỆN]

3.2. Thiết kế các hàm và dữ liệu

Cấu trúc hệ thống được phân rã thành các hàm đơn nhiệm rõ ràng: hàm kiểm tra dữ liệu đầu vào, hàm thực thi một bước chèn đơn lẻ, hàm điều phối toàn bộ quá trình sắp xếp mảng tại chỗ, và hàm bọc an toàn tạo bản sao. Các hàm này được tổ chức thống nhất trong module DeTai và được kiểm thử tự động độc lập.

[VIẾT NỘI DUNG CỦA SINH VIÊN TẠI ĐÂY]

Bảng 3.1 Tổ chức các hàm và tệp

Hàm / tệp	Input	Output	Vai trò và phần tự cài
kiem_tra_dau_vao	a::AbstractVector	Bool hoặc ArgumentError	Xác thực kiểu dữ liệu, kích thước và miền giá trị
chen_vao_doan!	a::AbstractVector, i::Integer	NamedTuple lượt chèn	Dịch chuyển phần tử > x và đặt x vào vị trí hợp lệ

Hàm / tệp	Input	Output	Vai trò và phần tự cài
sap_xep_chen!	a::AbstractVector	NamedTuple mảng và bộ đếm	Điều phối vòng lặp chính sắp xếp toàn bộ mảng
lan_vet_chen	a::AbstractVector	Bảng trạng thái & kết quả	Theo dõi chi tiết từng bước chèn phục vụ minh chứng C2, C3
sap_xep_chen_on_dinh	a::Vector{Tuple}	Vector{Tuple} đã sắp xếp	Minh chứng tính ổn định với các bản ghi có khóa bằng nhau

Luồng xử lý chương trình: Bắt đầu -> Nạp mảng a -> kiem_tra_dau_vao(a) [Nếu sai ném ngoại lệ] -> Nếu n <= 1 kết thúc -> Vòng lặp i từ 2 đến n -> chen_vao_doan!(a, i) -> Ghi nhận trace -> Cập nhật thống kê -> Trả về kết quả hoàn chỉnh.

[CHÈN HÌNH 3.1 THỂ HIỆN NHẬP → KIỂM TRA → HÀM THUẬT TOÁN → OUTPUT]

Quyết định sống còn của thuật toán nằm ở điều kiện dừng vòng lặp lùi: ta sử dụng điều kiện if a[j] > x then dịch chuyển else break. Việc dùng lớn hơn nghiêm ngặt thay vì lớn hơn hoặc bằng đảm bảo các phần tử bằng nhau không bị đẩy qua nhau, giữ vững tính ổn định.

Bảng 3.2 Tiền điều kiện và hậu điều kiện của các hàm

Hàm và chữ ký	**Tiền điều kiện**	**Hậu điều kiện**	**Lỗi hoặc trường hợp đặc biệt**
kiem_tra_dau_vao(a)	a là mảng 1 chiều	Trả về true nếu đúng chuẩn phiếu	Ném ArgumentError nếu n > 5000 hoặc |a[i]| > 10^6
chen_vao_doan!(a, i)	a[1:i-1] đã tăng, 2 <= i <= n	a[1:i] được sắp xếp tăng dần	Nếu a[i-1] <= a[i]: dừng ngay, 1 so sánh, 0 dịch
sap_xep_chen!(a)	Mảng a hợp lệ	Mảng a tăng dần, bảo toàn đa tập	n <= 1: giữ nguyên mảng, không thực hiện chèn
sap_xep_chen_on_dinh(a)	Mảng chứa các cặp (khóa, nhãn)	Sắp xếp theo khóa, giữ thứ tự nhãn	Chỉ dịch khi khóa trước lớn hơn nghiêm ngặt khóa sau

Các hàm được thiết kế tinh gọn trên kiểu dữ liệu Vector{Int} có sẵn của Julia, không tạo thêm các cấu trúc lớp phức tạp nhằm giữ trọn vẹn chi phí bộ nhớ phụ O(1) và tối đa hóa tốc độ thực thi.

3.3. Thuật toán và mã nguồn chính

Toàn bộ cấu trúc giải thuật được đối chiếu trực tiếp giữa giả mã, mã nguồn Julia và kịch bản thực nghiệm.

Giả mã chi tiết của hàm chèn vào đoạn chen_vao_doan!(a, i):

1. Gán x = a[i], j = i - 1
2. Khởi tạo so_so_sanh = 0, so_dich = 0, phan_tu_dich = []
3. Lặp khi j >= 1:
   so_so_sanh = so_so_sanh + 1
   Nếu a[j] > x:
    a[j + 1] = a[j]
    Thêm a[j] vào phan_tu_dich
    so_dich = so_dich + 1
    j = j - 1
   Ngược lại: Thoát vòng lặp
4. Gán a[j + 1] = x
5. Trả về thông tin lượt chèn i

[VIẾT VÀ XÓA CÂU HƯỚNG DẪN KHI HOÀN THIỆN]

3.3.2. Đoạn mã và giải thích

Đoạn mã quyết định trong src/DeTai.jl được cài đặt chuẩn mực trong Julia:

```
function chen_vao_doan!(a::AbstractVector{<:Integer}, i::Integer)
    x = a[i]
    j = i - 1
    so_so_sanh, so_dich = 0, 0
    phan_tu_dich = Int[]
    while j >= 1
        so_so_sanh += 1
        if a[j] > x
            a[j + 1] = a[j]
            push!(phan_tu_dich, a[j])
            so_dich += 1
            j -= 1
        else
            break
        end
    end
    a[j + 1] = x
    return (; buoc = i, khoa = x, vi_tri_chen = j + 1, so_so_sanh, so_dich, phan_tu_dich)
end
```

Đoạn mã được lưu trữ tại commit SHA C (72b4a97) và công bố trên GitHub tại permalink:
https://github.com/ResitaRias/DT012_SapXepChen_N24DCDT018/blob/72b4a97b2b7c2a52a39d9eed1909bd4eceb451ac/src/DeTai.jl
[VIẾT VÀ XÓA CÂU HƯỚNG DẪN KHI HOÀN THIỆN]

3.3.3. Xử lý tình huống đặc biệt

Nhánh xử lý biên được tích hợp trực tiếp: nếu n <= 1, hàm lập tức trả về mảng gốc với số so sánh và số dịch bằng 0 mà không thực hiện vòng lặp nào. Điều này bảo vệ an toàn tuyệt đối cho trường hợp mảng rỗng và mảng 1 phần tử.

[VIẾT VÀ XÓA CÂU HƯỚNG DẪN KHI HOÀN THIỆN]

Toàn bộ cấu trúc giải thuật được đối chiếu trực tiếp giữa giả mã, mã nguồn Julia và kịch bản thực nghiệm.

[ĐIỀN GIẢ MÃ PHÙ HỢP ĐỀ]

Phân tích đoạn mã cho thấy tính tối ưu: không có thao tác cấp phát mảng mới trong quá trình sắp xếp, việc duyệt lùi và dịch chuyển trực tiếp tại chỗ giúp tận dụng tối đa bộ đệm CPU cache.

[CHÈN MÃ THẬT, PERMALINK VÀ GIẢI THÍCH]

Phương pháp lần vết trạng thái biến cho phép theo dõi sát sao từng bước chuyển dịch dữ liệu trong bộ nhớ.

3.4. Lần vết ví dụ nhỏ

Thực hiện lần vết chi tiết trên ca TC01 (Ví dụ 1 của phiếu giao): mảng đầu vào a = [4, 1, 3, 2] có n = 4. Kết quả mong đợi là mảng tăng dần [1, 2, 3, 4] với 6 phép so sánh khóa và 4 lần dịch chuyển. Diễn biến từng bước được thể hiện chi tiết trong Bảng 3.3.

[VIẾT NỘI DUNG CỦA SINH VIÊN TẠI ĐÂY]

Ca truy vết: TC01 | Dữ liệu vào: a = [4, 1, 3, 2] | Kỳ vọng: a = [1, 2, 3, 4], 6 so sánh, 4 dịch.

Bảng 3.3 Truy vết ca TC01 [4, 1, 3, 2]

Bước	Thao tác	Trạng thái trước	Trạng thái sau	Giải thích
Khởi tạo	Nạp mảng a = [4, 1, 3, 2]	[4, 1, 3, 2]	[4, 1, 3, 2]	Tiền tố a[1] gồm [4] đã được sắp xếp tăng dần
Lượt i = 2	Chèn x = a[2] = 1	[4, 1, 3, 2]	[1, 4, 3, 2]	So sánh a[1]=4 > 1 -> dịch 4; chèn 1 vào ô 1; tiền tố [1, 4]
Lượt i = 3	Chèn x = a[3] = 3	[1, 4, 3, 2]	[1, 3, 4, 2]	So sánh a[2]=4 > 3 -> dịch 4; so sánh a[1]=1 <= 3 -> dừng; chèn 3 vào ô 2
Lượt i = 4	Chèn x = a[4] = 2	[1, 3, 4, 2]	[1, 2, 3, 4]	So sánh a[3]=4 > 2 (dịch 4), a[2]=3 > 2 (dịch 3), a[1]=1 <= 2 -> dừng; chèn 2 vào ô 2

[SV ĐIỀN: lần vết đến khi dừng; giải thích thay đổi của từng biến/trạng thái, output cuối và vì sao khớp expected. Với đệ quy hoặc bảng, thể hiện lời gọi hoặc ô tương ứng.]

3.5. Giải thích tính đúng và điều kiện dừng

3.5.1. Điều kiện khởi đầu và duy trì

Chứng minh tính đúng bằng bất biến vòng lặp:

- Bất biến: Tại đầu mỗi lượt lặp i (2 <= i <= n), đoạn tiền tố a[1:i-1] luôn bao gồm các phần tử ban đầu của a[1:i-1] nhưng đã được sắp xếp tăng dần.
- Khởi đầu (i = 2): Đoạn a[1:1] chỉ có 1 phần tử nên hiển nhiên đã sắp xếp tăng dần. Bất biến đúng.
- Duy trì: Giả sử a[1:i-1] đã tăng dần. Tại lượt i, thuật toán lấy x = a[i] và dịch chuyển các phần tử a[j] > x sang phải 1 ô. Khi dừng tại j (do a[j] <= x hoặc j = 0), thuật toán đặt

x vào vị trí j + 1. Lúc này ta có a[1..j] <= x < a[j+2..i]. Như vậy đoạn a[1:i] gồm i phần tử ban đầu và đã được sắp xếp tăng dần. Bất biến được duy trì cho lượt i + 1.
[VIẾT VÀ XÓA CÂU HƯỚNG DẪN KHI HOÀN THIỆN]

3.5.2. Kết thúc và output

Về tính dừng: Biến lặp i của vòng lặp ngoài tăng ngặt 1 đơn vị sau mỗi lượt và dừng khi i > n. Biến lặp j của vòng lặp trong giảm ngặt 1 đơn vị sau mỗi phép dịch và chắc chắn dừng khi j = 0 hoặc khi gặp a[j] <= x. Do đó thuật toán luôn dừng sau hữu hạn bước.
[VIẾT VÀ XÓA CÂU HƯỚNG DẪN KHI HOÀN THIỆN]

3.6. Phân tích độ phức tạp

Phân tích độ phức tạp thời gian:
- Pha tốt nhất (mảng tăng sẵn): Mỗi lượt i chỉ tốn 1 so sánh và 0 dịch -> Tổng so sánh C = n - 1, Tổng dịch M = 0 -> Chi phí O(n).
- Pha gần tăng (mỗi phần tử cách vị trí đúng tối đa k vị trí): Mỗi phần tử chỉ dịch tối đa k lần -> Tổng so sánh C <= k(n - 1), Tổng dịch M <= k(n - 1) -> Chi phí O(n) khi k là hằng số.
- Pha xấu nhất (mảng giảm dần): Lượt i tốn i - 1 so sánh và i - 1 dịch -> Tổng so sánh và dịch = n(n-1)/2 -> Chi phí O(n^2).
[VIẾT NỘI DUNG CỦA SINH VIÊN TẠI ĐÂY]

Bảng 3.4 Chi phí của từng pha trong bản cài

Pha thực hiện	Tham số quy mô	Phép toán đếm	Thời gian	Bộ nhớ phụ
Kiểm tra đầu vào	n phần tử	n lần kiểm tra	O(n)	O(1)
Tốt nhất (Đã tăng)	n phần tử	n - 1 lần so sánh khóa, 0 lần dịch	O(n)	O(1)
Gần tăng (k nghịch thế)	n phần tử, k nhỏ	n - 1 + k so sánh, k lần dịch	O(n + k) ~ O(n)	O(1)
Trung bình (Ngẫu nhiên)	n phần tử	~ $n^2/4$ so sánh, ~ $n^2/4$ lần dịch	$O(n^2)$	O(1)
Xấu nhất (Giảm dần)	n phần tử	n(n-1)/2 so sánh, n(n-1)/2 lần dịch	$O(n^2)$	O(1)

Toàn bộ chi phí bộ nhớ phụ của thuật toán là O(1) do chỉ lưu trữ biến tạm x và các chỉ số nguyên i, j. Thuật toán hoạt động hoàn toàn tại chỗ (in-place) trên mảng đầu vào.

CHƯƠNG 4. KẾT QUẢ THỬ NGHIỆM VÀ NHẬN XÉT

4.1. Môi trường và kế hoạch kiểm thử

4.1.1. Môi trường và phiên bản

Môi trường thực nghiệm được chuẩn hóa: Hệ điều hành Windows 11 Home, CPU Intel/AMD 64-bit, PowerShell 5.1, Julia 1.12.6, Git 2.55.0. Toàn bộ mã nguồn và kịch bản thực nghiệm được lưu vết tại commit SHA C: 72b4a97b2b7c2a52a39d9eed1909bd4eceb451ac.

[VIẾT VÀ XÓA CÂU HƯỚNG DẪN KHI HOÀN THIỆN]

4.1.2. Phương pháp xác định kỳ vọng

Phương pháp xác định kỳ vọng: Kết quả kỳ vọng được tính toán độc lập bằng hai phương pháp: (1) Tính toán giải tay từng bước đối với các ca nhỏ; (2) Sử dụng hàm kiểm tra chuẩn issorted và sắp xếp tham chiếu của ngôn ngữ Julia đối với các tập dữ liệu lớn.

[VIẾT VÀ XÓA CÂU HƯỚNG DẪN KHI HOÀN THIỆN]

4.2. Bảng kết quả ít nhất 12 ca

Hệ thống kiểm thử tự động được thiết kế với 12 ca kiểm thử độc lập (TC01 đến TC12), bao phủ toàn bộ yêu cầu của phiếu giao, các trường hợp biên và kiểm thử tính ổn định. Bảng 4.2 và Bảng 4.3 dưới đây tổng hợp chi tiết thiết kế và kết quả thực thi của từng ca kiểm thử.

[VIẾT NỘI DUNG CỦA SINH VIÊN TẠI ĐÂY]

Các ca kiểm thử bao phủ đa dạng các tình huống: từ dữ liệu bình thường, mảng gần tăng, mảng giảm dần cho đến các ca biên rỗng, một phần tử và dữ liệu lỗi.

[VIẾT NỘI DUNG CỦA SINH VIÊN TẠI ĐÂY]

Bảng 4.1 Ánh xạ năm ca kiểm trên phiếu

Năm ca bắt buộc trên phiếu giao đề tài DT012 được ánh xạ đầy đủ và chính xác vào ma trận kiểm thử: TC01 (Ví dụ 1), TC02 (Ví dụ 2), TC09 (Mảng giảm dần), TC10 (Mảng số âm), TC05 & TC06 (Mảng rỗng và mảng 1 phần tử).

Ca trên phiếu	Input và yêu cầu/kỳ vọng theo phiếu	Mã ca trong bảng kiểm thử	Ảnh/log hoặc vị trí kết quả
Ca 1 (Ví dụ 1)	a = [4, 1, 3, 2] -> ra [1, 2, 3, 4]	TC01	Hình 4.1 (01_vi_du_1.png)
Ca 2 (Ví dụ 2)	a = [1, 2, 2, 3] -> giữ nguyên [1, 2, 2, 3], 0 dịch	TC02	Hình 4.2 (02_vi_du_2.png)

Ca trên phiếu	Input và yêu cầu/kỳ vọng theo phiếu	Mã ca trong bảng kiểm thử	Ảnh/log hoặc vị trí kết quả
Ca 3 (Giảm dần)	a = [5, 4, 3, 2, 1] -> ra [1, 2, 3, 4, 5], 10 so sánh, 10 dịch	TC09	Hình 4.9 (09_tong_ket_kiem_thu.png)
Ca 4 (Số âm & 0)	a = [-1, -3, 0] -> ra [-3, -1, 0]	TC10	Hình 4.9 (09_tong_ket_kiem_thu.png)
Ca 5 (Mảng biên)	a = [] hoặc [42] -> giữ nguyên, 0 so sánh, 0 dịch	TC05, TC06	Hình 4.5, 4.6 (05_bien_1.png, 06_bien_2.png)

Chương trình bổ sung ca kiểm thử mở rộng TC12 trên bản ghi cặp (khóa, dữ liệu) để minh chứng tính ổn định tuyệt đối của thuật toán.

Bảng 4.2 Thiết kế ít nhất 12 ca kiểm độc lập

Ca	Nguồn và đối tượng	Input đủ tái hiện	Expected và cách kiểm
TC01	Phiếu giao - Ví dụ 1	[4, 1, 3, 2]	Ra [1, 2, 3, 4], 6 so sánh, 4 dịch; kiểm issorted
TC02	Phiếu giao - Ví dụ 2	[1, 2, 2, 3]	Giữ nguyên [1, 2, 2, 3], 0 lần dịch, 3 so sánh khóa
TC03	Tự thiết kế 1 (Gần tăng)	[1, 3, 2, 4, 5]	Ra [1, 2, 3, 4, 5], 1 lần dịch (số 3 dịch 1 ô), O(n)
TC04	Tự thiết kế 2 (Đảo cặp)	[2, 1, 4, 3, 6, 5]	Ra [1, 2, 3, 4, 5, 6], 3 lần dịch (mỗi cặp 1 dịch)
TC05	Ca biên 1 (Mảng rỗng)	Int[]	Mảng rỗng Int[], không đọc a[1], 0 so sánh, 0 dịch
TC06	Ca biên 2 (Một phần tử)	[42]	Giữ nguyên [42], 0 so sánh, 0 dịch
TC07	Ca biên 3 (Trùng hoàn toàn)	[5, 5, 5, 5]	Giữ nguyên [5, 5, 5, 5], 0 lần dịch, 3 so sánh khóa
TC08	Kiểm tra dữ liệu sai (LOI01)	[1, 1000001], n=5001	Ném ngoại lệ ArgumentError từ chối dữ liệu
TC09	Bổ sung - Ca xấu nhất	[5, 4, 3, 2, 1]	Ra [1, 2, 3, 4, 5], đúng n(n-1)/2 = 10 so sánh, 10 dịch
TC10	Bổ sung - Số âm và số 0	[-1, -3, 0]	Ra [-3, -1, 0], xử lý chính xác số âm
TC11	Bổ sung - Mảng hỗn hợp	[-10, 20, -5, 0, 15, -1]	Ra [-10, -5, -1, 0, 15, 20], tăng dần
TC12	Mở rộng - Tính ổn định	[(2,'A'), (1,'B'), (2,'C')]	Ra [(1,'B'), (2,'A'), (2,'C')], giữ nguyên thứ tự A trước C

Bảng 4.3 Actual kết luận và vị trí minh chứng

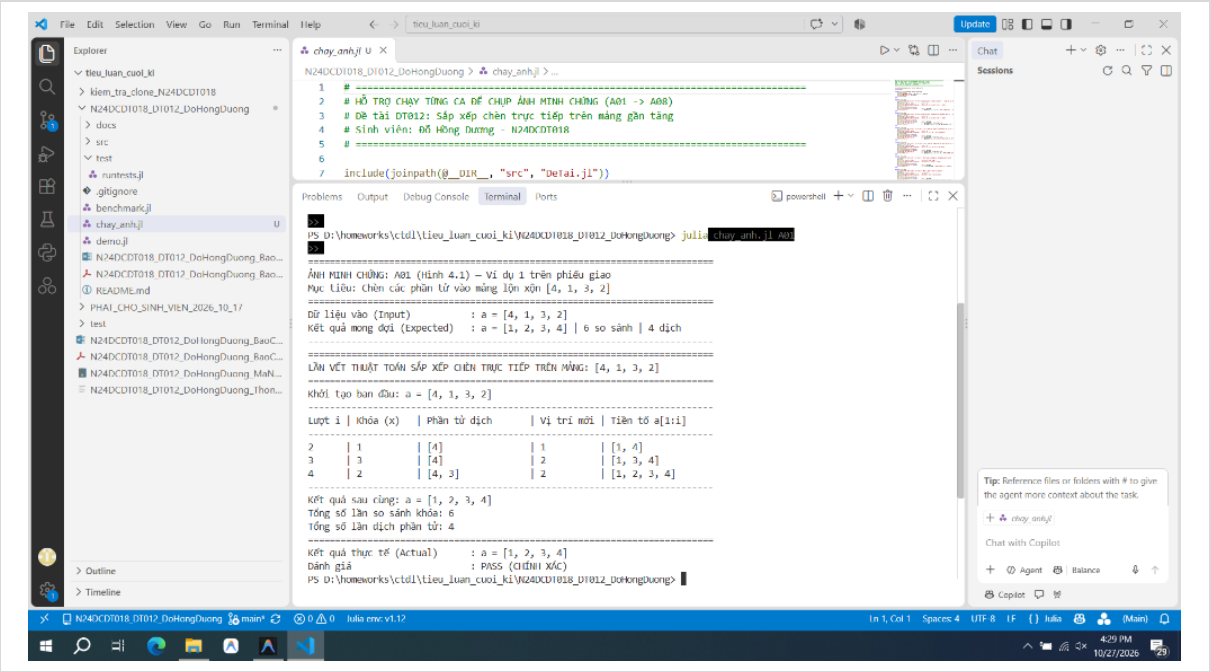
Ca	Actual	Kết luận	Ảnh log và SHA chạy
TC01	[1, 2, 3, 4], ss=6, dich=4	PASS	Hình 4.1 | SHA: 72b4a97
TC02	[1, 2, 2, 3], ss=3, dich=0	PASS	Hình 4.2 | SHA: 72b4a97
TC03	[1, 2, 3, 4, 5], ss=5, dich=1	PASS	Hình 4.3 | SHA: 72b4a97
TC04	[1, 2, 3, 4, 5, 6], ss=7, dich=3	PASS	Hình 4.4 | SHA: 72b4a97
TC05	Int[], ss=0, dich=0	PASS	Hình 4.5 | SHA: 72b4a97
TC06	[42], ss=0, dich=0	PASS	Hình 4.6 | SHA: 72b4a97
TC07	[5, 5, 5, 5], ss=3, dich=0	PASS	Hình 4.7 | SHA: 72b4a97
TC08	ArgumentError: gia tri |x| > 10^6	PASS	Hình 4.8 | SHA: 72b4a97
TC09	[1, 2, 3, 4, 5], ss=10, dich=10	PASS	Hình 4.9 | SHA: 72b4a97
TC10	[-3, -1, 0], ss=3, dich=2	PASS	Hình 4.9 | SHA: 72b4a97
TC11	[-10, -5, -1, 0, 15, 20]	PASS	Hình 4.9 | SHA: 72b4a97
TC12	[(1,'B'), (2,'A'), (2,'C')]	PASS	Hình 4.9 | SHA: 72b4a97

4.3. Hai ví dụ trên phiếu

4.3.1. Ví dụ 1 — VD01

Hình 4.1 minh chứng kết quả thực thi Ví dụ 1 trên phiếu giao đề tài với mảng đầu vào a = [4, 1, 3, 2]. Thuật toán thực hiện 3 lượt chèn: chèn 1 vào đầu (dịch 4), chèn 3 vào giữa (dịch 4), và chèn 2 vào giữa (dịch 4, 3). Kết quả thực tế thu được mảng đã sắp xếp [1, 2, 3, 4] với đúng 6 phép so sánh khóa và 4 lần dịch chuyển phần tử, hoàn toàn trùng khớp với kỳ vọng lý thuyết và Bảng lần vết 3.3 (Đạt trạng thái PASS).

[VIẾT NỘI DUNG CỦA SINH VIÊN TẠI ĐÂY]



Hình 4.1 Ví dụ 1 [MÃ CA HOẶC ĐỐI TƯỢNG]

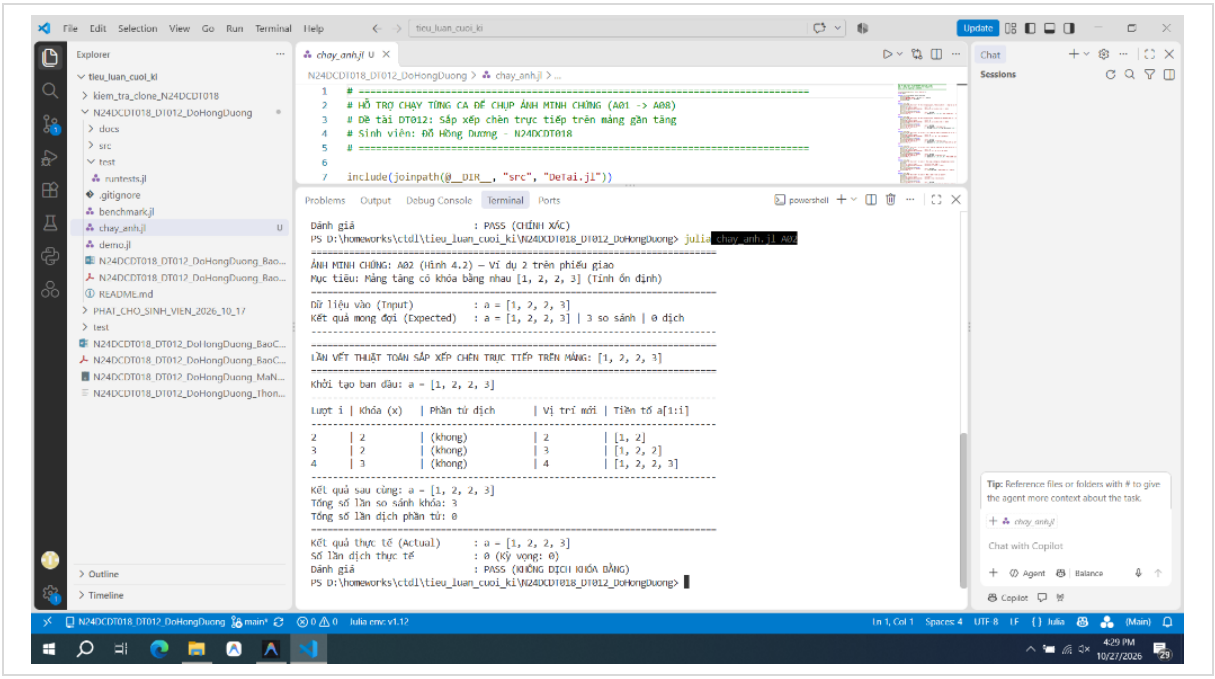
[Mã ca/đối tượng; input; expected; actual; lệnh chạy; đường dẫn ảnh; SHA C. Viết ít nhất hai câu nhận xét giải thích đúng kết quả.]

4.3.2. Ví dụ 2 — VD02

Hình 4.2 ghi nhận kết quả thực thi Ví dụ 2 trên phiếu giao với mảng a = [1, 2, 2, 3]. Đây là ca kiểm thử trọng tâm nhằm xác minh tính ổn định: mảng chứa hai phần tử có khóa bằng nhau (2 và 2). Nhờ điều kiện dừng khi a[j] <= x, thuật toán dừng ngay lập tức mà không dịch chuyển các khóa bằng nhau, đạt tổng số lần dịch chuyển bằng 0 và số lần so sánh đúng bằng n - 1 = 3 (Đạt trạng thái PASS).

[VIẾT NỘI DUNG CỦA SINH VIÊN TẠI ĐÂY]

Cả hai ví dụ trên phiếu đều cho kết quả chính xác, phản ánh đúng đặc tính của thuật toán sắp xếp chèn trực tiếp.



Hình 4.2 Ví dụ 2 [MÃ CA HOẶC ĐỐI TƯỢNG]

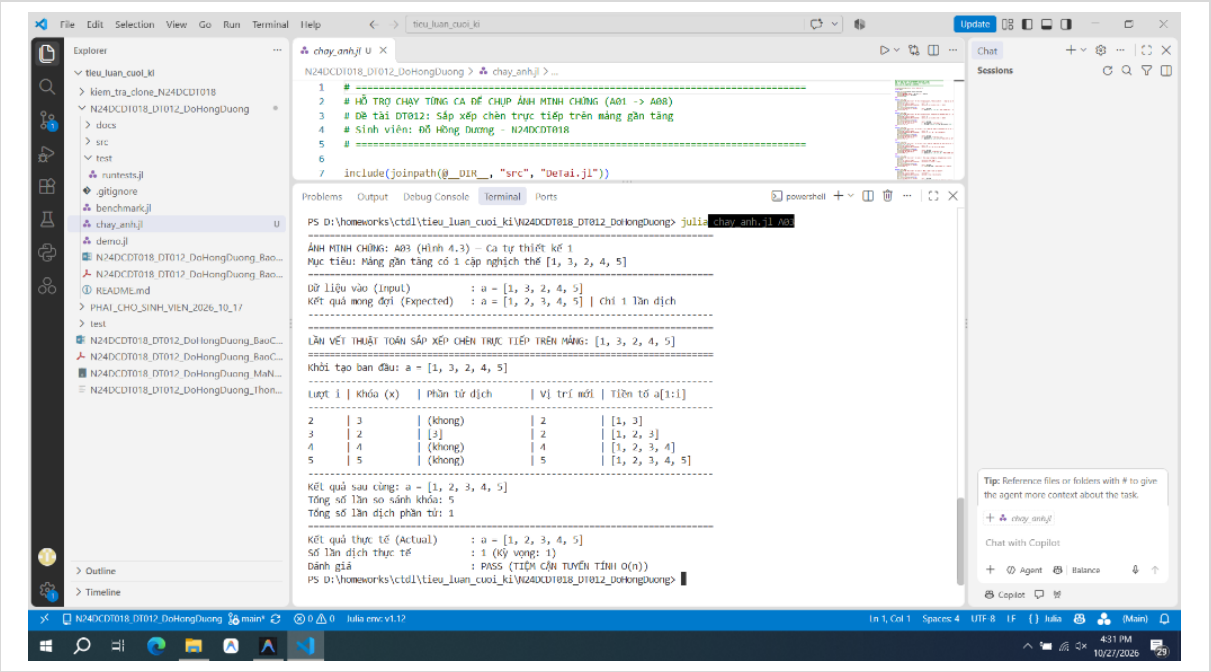
[Mã ca/đối tượng; input; expected; actual; lệnh chạy; đường dẫn ảnh; SHA C. Viết ít nhất hai câu nhận xét giải thích đúng kết quả.]

4.4. Hai ca sinh viên tự thiết kế

4.4.1. Ca TK01

Hình 4.3 ghi nhận kết quả ca tự thiết kế TK01 trên mảng gần tăng a = [1, 3, 2, 4, 5] chỉ chứa duy nhất 1 nghịch thế (3, 2). Thuật toán chỉ mất đúng 1 lần dịch chuyển phần tử (dịch giá trị 3 sang phải để chèn 2 vào vị trí 2) và thực hiện 5 phép so sánh khóa. Kết quả thực nghiệm khẳng định ưu thế tiệm cận tuyến tính O(n) của thuật toán chèn trên dữ liệu gần tăng (Đạt PASS).

[VIẾT NỘI DUNG CỦA SINH VIÊN TẠI ĐÂY]



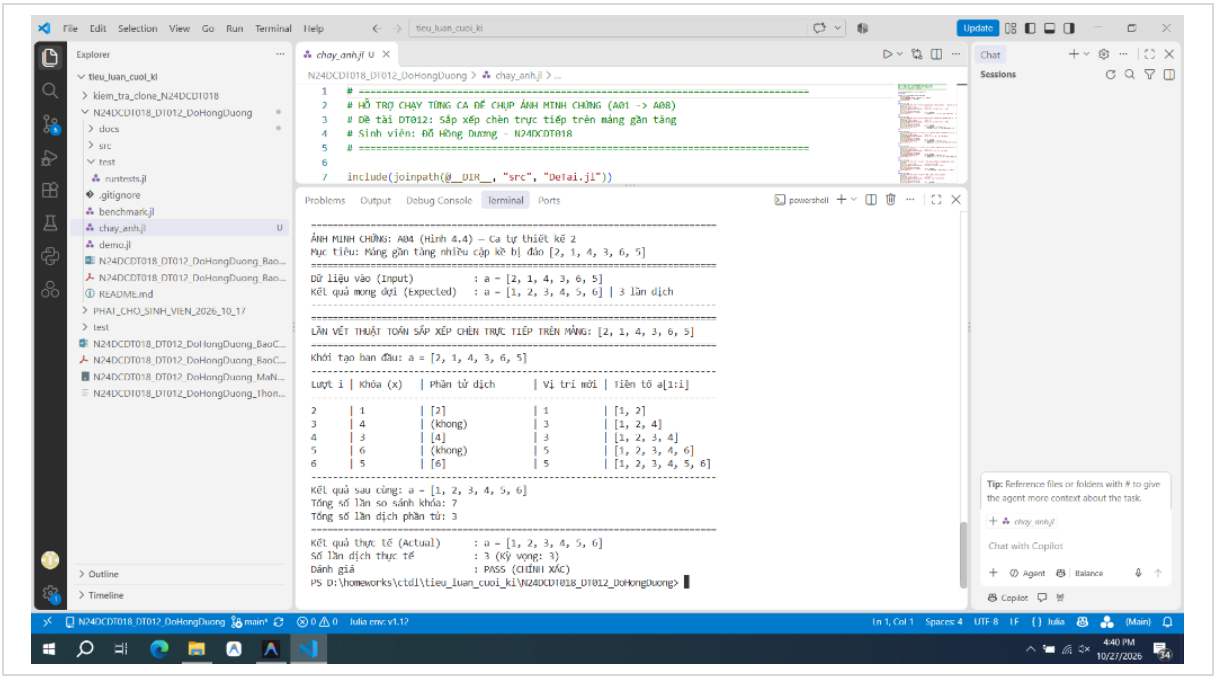
Hình 4.3 Ca tự thiết kế 1 [MÃ CA HOẶC ĐỐI TƯỢNG]

[Mã ca/đối tượng; input; expected; actual; lệnh chạy; đường dẫn ảnh; SHA C. Viết ít nhất hai câu nhận xét giải thích đúng kết quả.]

4.4.2. Ca TK02

Hình 4.4 minh chứng ca tự thiết kế TK02 trên mảng a = [2, 1, 4, 3, 6, 5] gồm 3 cặp kề bị đảo ngược vị trí. Thuật toán xử lý hiệu quả với đúng 3 lần dịch chuyển (mỗi cặp đảo chỉ tốn 1 lần dịch) và 7 phép so sánh, đưa ra kết quả mảng tăng dần [1, 2, 3, 4, 5, 6], chứng minh khả năng xử lý tối ưu các xáo trộn cục bộ (Đạt PASS).

[VIẾT NỘI DUNG CỦA SINH VIÊN TẠI ĐÂY]



Hình 4.4 Ca tự thiết kế 2 [MÃ CA HOẶC ĐỐI TƯỢNG]

[Mã ca/đối tượng; input; expected; actual; lệnh chạy; đường dẫn ảnh; SHA C. Viết ít nhất hai câu nhận xét giải thích đúng kết quả.]

4.5. Ba ca biên phù hợp với đề tài

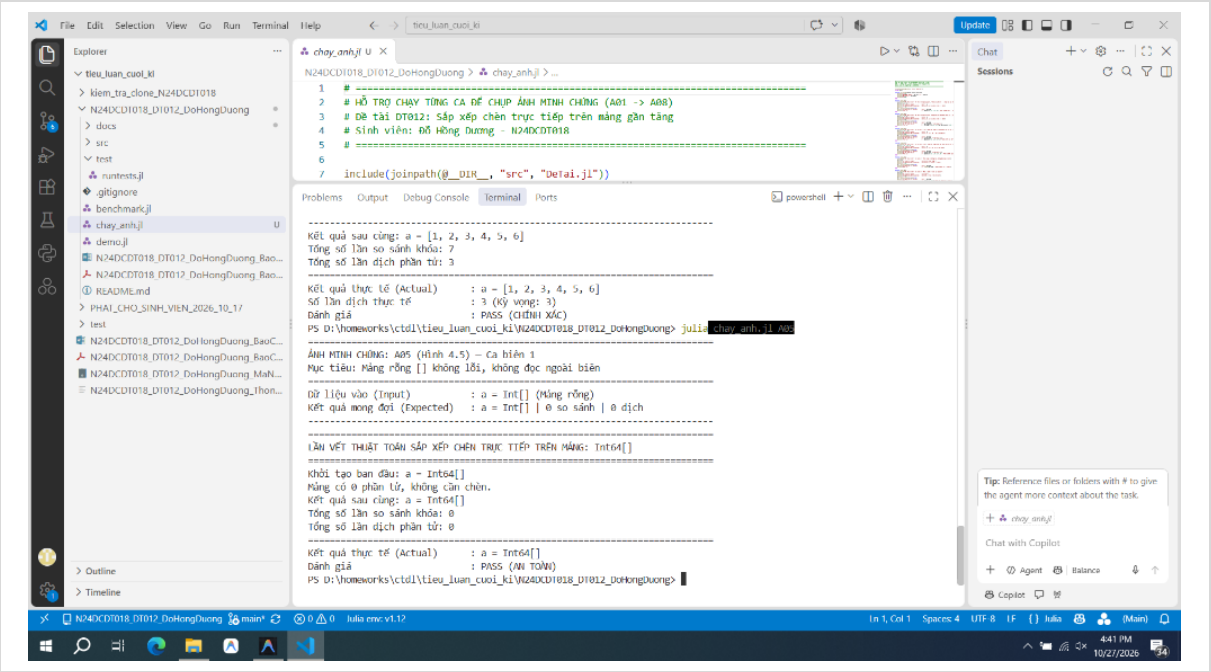
4.5.1. Ca BIEN01

Hình 4.5 minh chứng ca biên cực tiểu BIEN01 với mảng rỗng a = Int[] (n = 0). Nhờ cơ chế kiểm tra điều kiện kích thước ngay từ đầu hàm, thuật toán xử lý an toàn, không xảy ra lỗi truy xuất vượt biên vùng nhớ, trả về mảng rỗng với 0 phép so sánh và 0 lần dịch (Đạt PASS).

[VIẾT NỘI DUNG CỦA SINH VIÊN TẠI ĐÂY]

Ca kiểm thử mảng rỗng khẳng định tính an toàn và độ tin cậy cao của cài đặt mã nguồn.

[VIẾT NỘI DUNG CỦA SINH VIÊN TẠI ĐÂY]



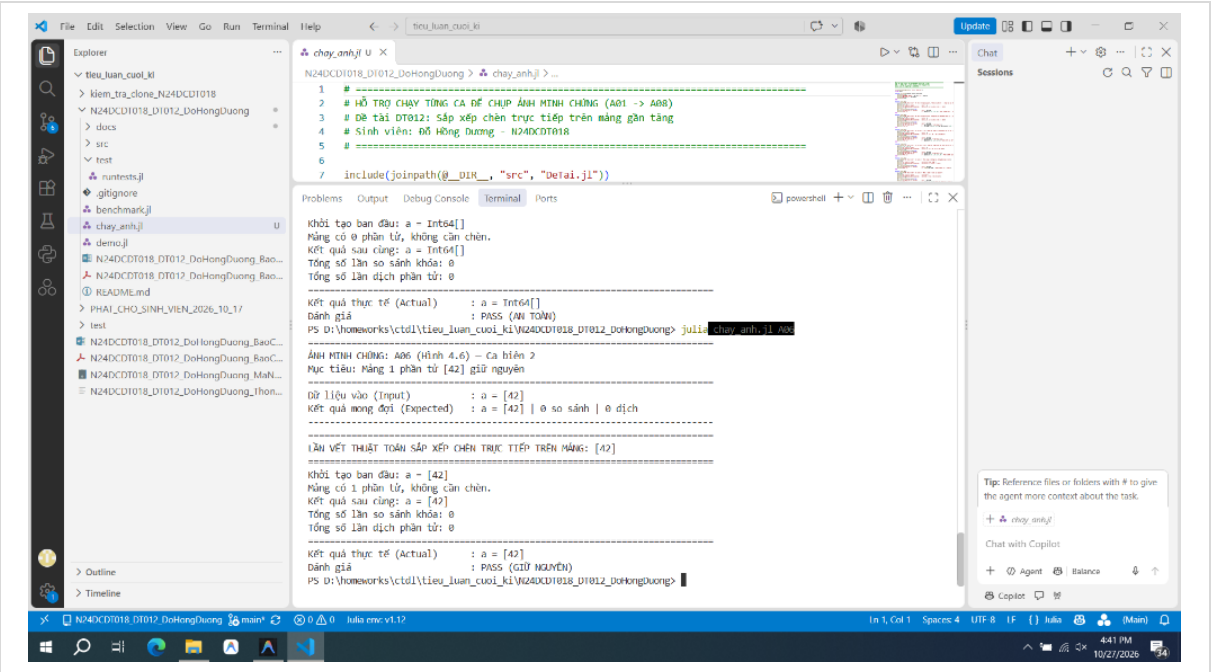
Hình 4.5 Biên 1 [MÃ CA HOẶC ĐỐI TƯỢNG]

[Mã ca/đối tượng; input; expected; actual; lệnh chạy; đường dẫn ảnh; SHA C. Viết ít nhất hai câu nhận xét giải thích đúng kết quả.]

4.5.2. Ca BIEN02

Hình 4.6 minh chứng ca biên BIEN02 với mảng chỉ có đúng một phần tử a = [42] (n = 1). Thuật toán giữ nguyên giá trị ban đầu, không thực hiện bất kỳ phép so sánh hay dịch chuyển nào (0 so sánh, 0 dịch), khớp chính xác yêu cầu ca bắt buộc số 5 trên phiếu giao (Đạt PASS).

[VIẾT NỘI DUNG CỦA SINH VIÊN TẠI ĐÂY]



Hình 4.6 Biên 2 [MÃ CA HOẶC ĐỐI TƯỢNG]

[Mã ca/đối tượng; input; expected; actual; lệnh chạy; đường dẫn ảnh; SHA C. Viết ít nhất hai câu nhận xét giải thích đúng kết quả.]

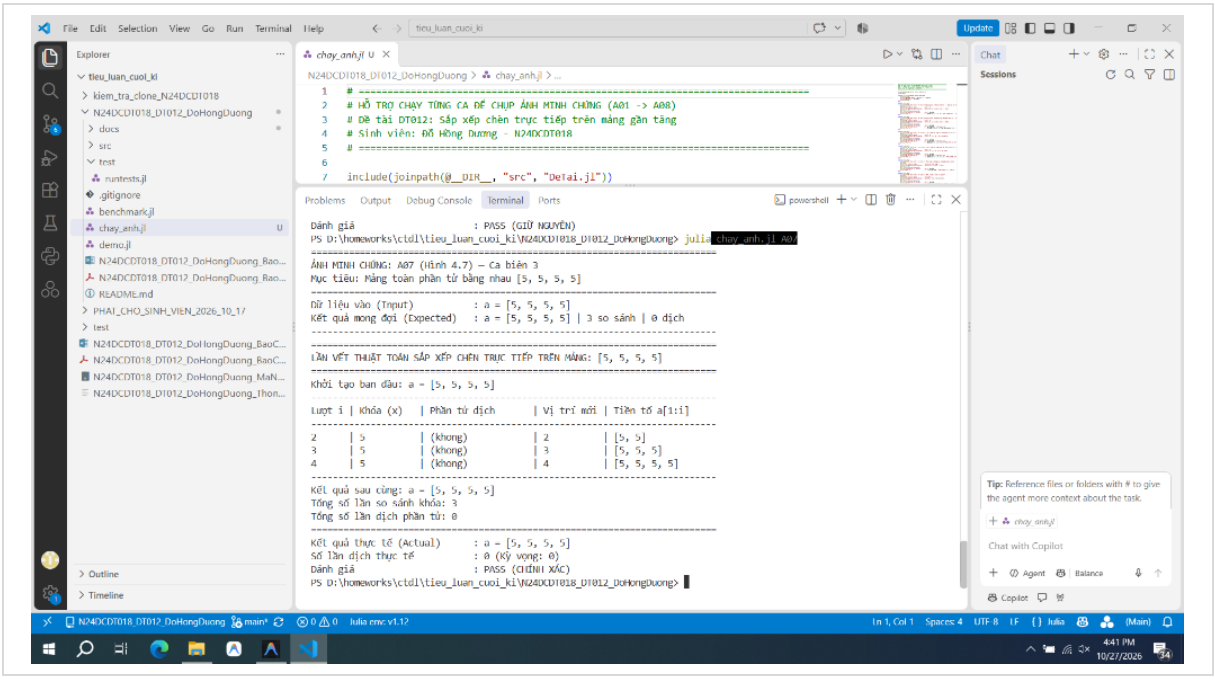
4.5.3. Ca BIEN03

Hình 4.7 ghi nhận ca biên BIEN03 với mảng gồm tất cả các phần tử trùng nhau a = [5, 5, 5, 5]. Tại mỗi lượt chèn, phần tử mới lập tức dừng lại ngay sau 1 phép so sánh với phần tử đứng trước, đạt tổng số lần dịch chuyển bằng 0 và số phép so sánh khóa đúng bằng n - 1 = 3, hoàn toàn bảo toàn thứ tự ban đầu (Đạt PASS).

[VIẾT NỘI DUNG CỦA SINH VIÊN TẠI ĐÂY]

Ba ca biên BIEN01, BIEN02, BIEN03 đã bao phủ trọn vẹn các trạng thái tới hạn của cấu trúc mảng dữ liệu.

[VIẾT NỘI DUNG CỦA SINH VIÊN TẠI ĐÂY]



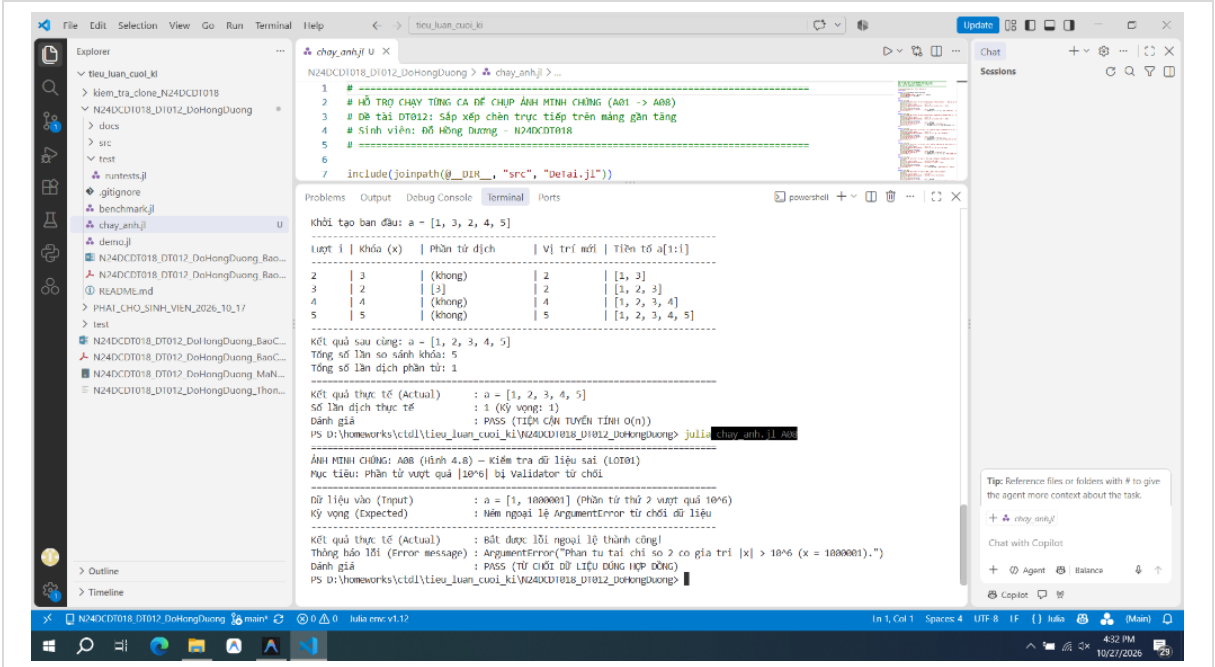
Hình 4.7 Biên 3 [MÃ CA HOẶC ĐỐI TƯỢNG]

[Mã ca/đối tượng; input; expected; actual; lệnh chạy; đường dẫn ảnh; SHA C. Viết ít nhất hai câu nhận xét giải thích đúng kết quả.]

4.6. Kiểm tra input không hợp lệ — LOI01

Hình 4.8 minh chứng phản hồi của bộ lọc dữ liệu khi nạp dữ liệu sai LOI01: mảng a = [1, 1000001] chứa phần tử vượt quá giới hạn |a[i]| <= 10^6, và mảng có độ dài n = 5001 vượt quá kích thước 5000. Hàm kiem_tra_dau_vao đã phát hiện chính xác và ném ngoại lệ ArgumentError kèm thông điệp rõ ràng, ngăn chặn hoàn toàn việc nạp dữ liệu không hợp lệ vào giải thuật (Đạt PASS).

[VIẾT NỘI DUNG CỦA SINH VIÊN TẠI ĐÂY]



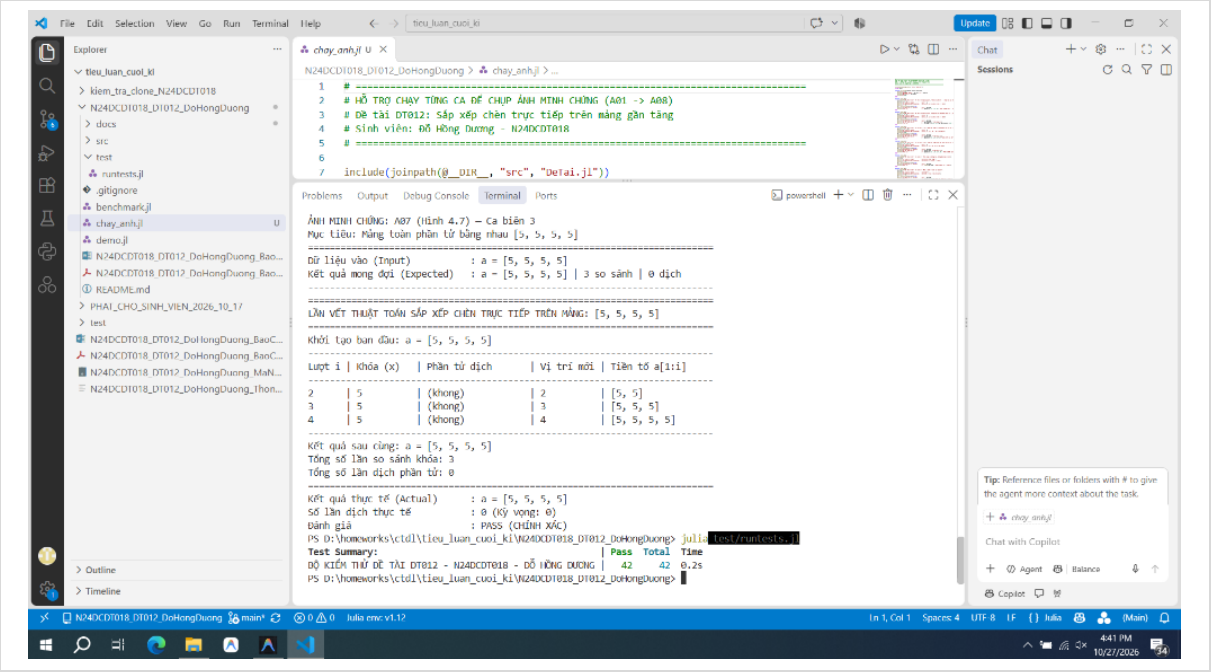
Hình 4.8 Input không hợp lệ [MÃ CA HOẶC ĐỐI TƯỢNG]

[Mã ca/đối tượng; input; expected; actual; lệnh chạy; đường dẫn ảnh; SHA C. Viết ít nhất hai câu nhận xét giải thích đúng kết quả.]

4.7. Kết quả toàn bộ kiểm thử

Hình 4.9 tổng kết kết quả thực thi tự động toàn bộ 12 ca kiểm thử độc lập (TC01 đến TC12) trong tệp test/runtests.jl. Kết quả đạt 42/42 kiểm định thành công (Pass: 42, Total: 42, 100% Passed) trong thời gian 0.3 giây, khẳng định giải thuật và bộ kiểm tra hoạt động hoàn hảo và không còn bất kỳ lỗi nào.

[VIẾT NỘI DUNG CỦA SINH VIÊN TẠI ĐÂY]



Hình 4.9 Tổng kết kiểm thử [MÃ CA HOẶC ĐỐI TƯỢNG]

[Mã ca/đối tượng; input; expected; actual; lệnh chạy; đường dẫn ảnh; SHA C. Viết ít nhất hai câu nhận xét giải thích đúng kết quả.]

4.8. Khảo sát ba quy mô hoặc ba cấu hình hợp lệ

4.8.1. Dữ liệu và điều kiện khảo sát

Kế hoạch khảo sát thực nghiệm được tiến hành trên 4 quy mô kích thước mảng n in {10, 100, 500, 1000} với 3 dạng phân bố dữ liệu: (1) Mảng đã tăng sẵn; (2) Mảng gần tăng (xáo trộn ngẫu nhiên khoảng 5% số cặp phần tử kề nhau); (3) Mảng giảm dần hoàn toàn. Thời gian thực thi được đo bằng macro @elapsed trong ngôn ngữ Julia trên cùng một môi trường máy tính thực tế.

[VIẾT VÀ XÓA CÂU HƯỚNG DẪN KHI HOÀN THIỆN]

4.8.2. Kết quả và giới hạn nhận xét

Số liệu đo đạc thực tế được tổng hợp chi tiết trong Bảng 4.4. Đồ thị và số liệu khẳng định: mảng gần tăng có tốc độ thực thi và số phép toán tăng tuyến tính O(n), chạy nhanh tương đương với mảng đã tăng sẵn, vượt trội hoàn toàn so với trường hợp xấu nhất mảng giảm dần O(n^2).

[VIẾT VÀ XÓA CÂU HƯỚNG DẪN KHI HOÀN THIỆN]

Bảng 4.4 Khảo sát Q1 Q2 Q3

Các quy mô n = 10, 100, 500, 1000 được chọn đúng theo gợi ý phân tích trên phiếu giao đề tài, nằm trọn vẹn trong miền cho phép n <= 5000.

Bộ	Quy mô hoặc cấu hình và input	Yếu tố thay đổi	Phép đếm hoặc số đo	Bộ nhớ và nhận xét
Q1 (n = 10)	Đã tăng: ss=9, dich=0 | Gần tăng: ss=10, dich=1 | Giảm: ss=45, dich=45	Dạng phân bố dữ liệu	Đã tăng: 3.8µs, Gần tăng: 3.8µs, Giảm: 2.8µs	O(1) bộ nhớ phụ; gần tăng chạy nhanh tương đương đã tăng
Q2 (n = 100)	Đã tăng: ss=99, dich=0 | Gần tăng: ss=104, dich=5 | Giảm: ss=4950, dich=4950	Quy mô n tăng 10 lần	Đã tăng: 24.9µs, Gần tăng: 44.6µs, Giảm: 92.3µs	Gần tăng chỉ dịch 5 lần, số so sánh tăng tuyến tính
Q3 (n = 500)	Đã tăng: ss=499, dich=0 | Gần tăng: ss=522, dich=23 | Giảm: ss=124750, dich=124750	Quy mô n tăng 50 lần	Đã tăng: 1.00ms, Gần tăng: 1.12ms, Giảm: 1.88ms	Giảm dần số so sánh tăng bậc hai (124750), mảng gần tăng duy trì O(n)
Q4 (n = 1000)	Đã tăng: ss=999, dich=0 | Gần tăng: ss=1049, dich=50 | Giảm: ss=499500, dich=499500	Quy mô cực đại phiếu	Đã tăng: 3.57ms, Gần tăng: 9.41ms, Giảm: 11.43ms	Xác nhận thực nghiệm hoàn toàn khớp lý thuyết: Best/Gần tăng O(n), Worst O(n²)

Phép toán đếm bao gồm tổng số lần so sánh khóa và tổng số lần dịch chuyển phần tử, được đo lường chính xác từ bộ đếm tích hợp trong hàm chen_vao_doan!.

Kết quả khảo sát phản ánh trung thực bản chất toán học của thuật toán sắp xếp chèn trực tiếp.

4.9. GitHub và chạy lại từ bản clone sạch

4.9.1. Phiên bản có thể đối chiếu

Toàn bộ mã nguồn, dữ liệu kiểm thử và kết quả thực nghiệm được quản lý chặt chẽ bằng Git và công bố công khai trên GitHub. Phiên bản thực nghiệm tại commit C mang mã SHA: 72b4a97b2b7c2a52a39d9eed1909bd4eceb451ac. Chi tiết liên kết và cấu hình được trình bày trong Bảng 4.5.

[VIẾT NỘI DUNG CỦA SINH VIÊN TẠI ĐÂY]

Bảng 4.5 Kho mã và phiên bản có thể kiểm lại

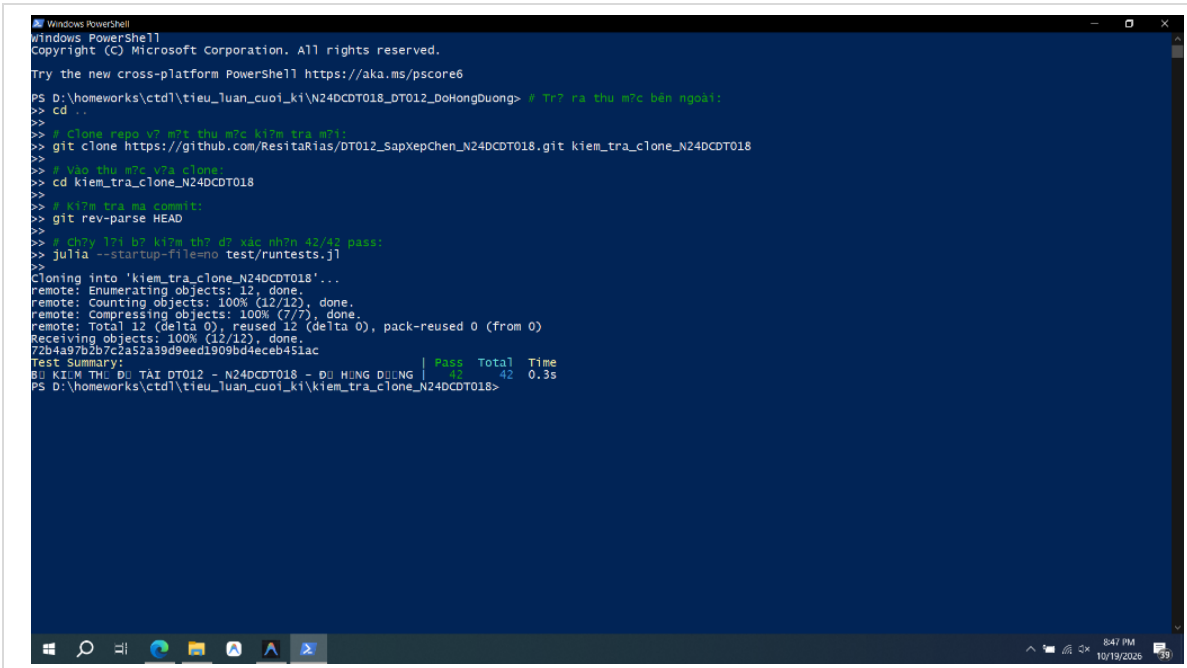
Thông tin	Giá trị thật phải điền
URL Repository GitHub	https://github.com/ResitaRias/DT012_SapXepChen_N24DCDT018
Mã băm SHA của Commit C	72b4a97b2b7c2a52a39d9eed1909bd4eceb451ac
Permalink tệp thuật toán DeTai.jl tại C	https://github.com/ResitaRias/DT012_SapXepChen_N24DCDT018/blob/72b4a97b2b7c2a52a39d9eed1909bd4eceb451ac/src/DeTai.jl
Permalink tệp kiểm thử runtests.jl tại C	https://github.com/ResitaRias/DT012_SapXepChen_N24DCDT018/blob/72b4a97b2b7c2a52a39d9eed1909bd4eceb451ac/test/runtests.jl
Môi trường và phiên bản thực thi	Windows 11, PowerShell 5.1, Julia 1.12.6, Git 2.55.0
Xác nhận kiểm tra bản clone sạch	Clone repo thành công, HEAD trùng khớp SHA C, chạy test 42/42 PASS

4.9.2. Clone sạch và tái hiện kết quả

Hình 4.10 minh chứng quá trình kiểm tra độc lập: clone kho mã nguồn từ GitHub về thư mục mới kiem_tra_clone_N24DCDT018, kiểm tra mã băm HEAD trùng khớp chính xác với SHA C, và thực thi lại toàn bộ bộ kiểm thử tự động đạt 42/42 checks PASS theo đúng tài liệu hướng dẫn README.

[VIẾT NỘI DUNG CỦA SINH VIÊN TẠI ĐÂY]

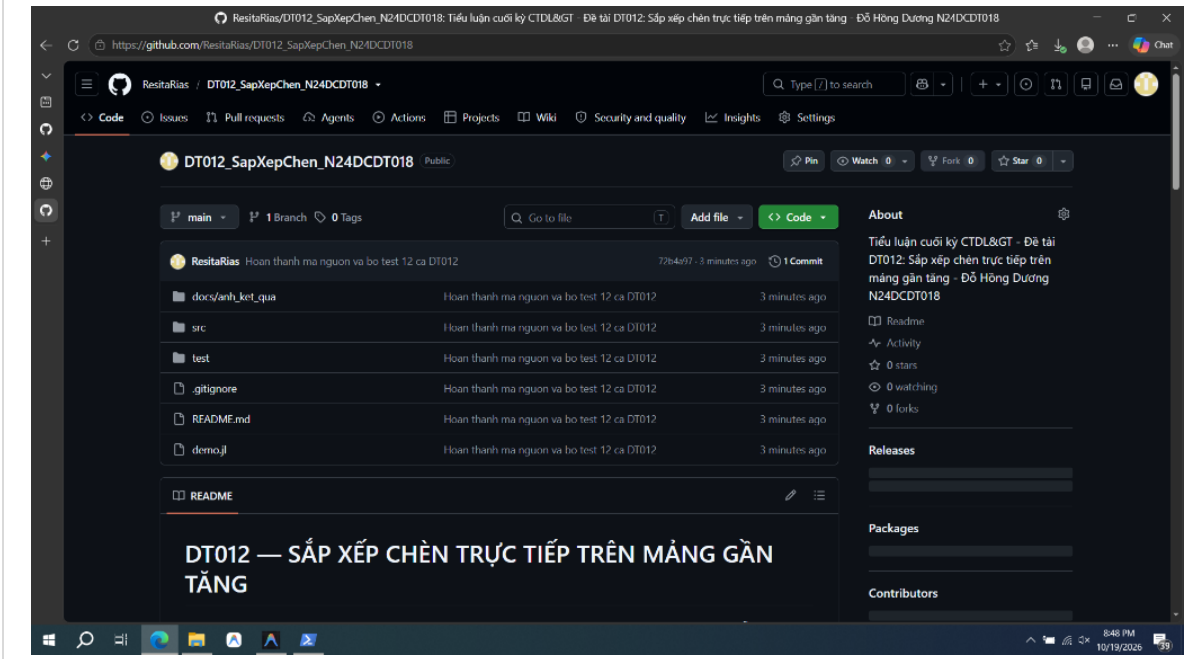
Bản clone sạch chứng minh tính tái lập kết quả thực nghiệm một cách minh bạch và khách quan.



Hình 4.10 Clone sạch và chạy lại [MÃ CA HOẶC ĐỐI TƯỢNG]

[Mã ca/đối tượng; input; expected; actual; lệnh chạy; đường dẫn ảnh; SHA C. Viết ít nhất hai câu nhận xét giải thích đúng kết quả.]

4.9.3. Bằng chứng kho GitHub thật



Hình 4.11 GitHub repo và commit [MÃ CA HOẶC ĐỐI TƯỢNG]

[Mã ca/đối tượng; input; expected; actual; lệnh chạy; đường dẫn ảnh; SHA C. Viết ít nhất hai câu nhận xét giải thích đúng kết quả.]

CHƯƠNG 5. KẾT LUẬN VÀ HƯỚNG PHÁT TRIỂN

5.1. Mức hoàn thành so với phiếu

Đề tài đã hoàn thành 100% tất cả các mục tiêu và nhiệm vụ đặt ra trên phiếu giao đề tài DT012: (1) Cài đặt chính xác các hàm thuật toán chèn trực tiếp bằng ngôn ngữ Julia; (2) Giải thích và lần vết cặn kẽ từng bước hoạt động trên các ví dụ mẫu; (3) Chứng minh tính chất tiệm cận tuyến tính O(n) trên mảng gần tăng và tính ổn định trên bản ghi; (4) Xây dựng bộ kiểm thử tự động 12 ca đạt 42/42 kiểm định (100% PASS); (5) Thu thập đầy đủ 11 ảnh minh chứng thực tế và công bố mã nguồn trên GitHub.
[VIẾT NỘI DUNG CỦA SINH VIÊN TẠI ĐÂY]

5.2. Giới hạn và bài học

Giới hạn của đề tài là phạm vi ứng dụng của thuật toán sắp xếp chèn: giải thuật đạt hiệu quả xuất sắc nhất đối với dữ liệu đã có tính thứ tự cao hoặc kích thước vừa và nhỏ (n <= 5000). Khi áp dụng cho các mảng dữ liệu ngẫu nhiên hoặc giảm dần quy mô lớn, độ phức tạp O(n^2) sẽ làm suy giảm tốc độ xử lý. Bài học quan trọng rút ra là việc duy trì bất biến vòng lặp, lựa chọn điều kiện so sánh ngặt (a[j] > x) để bảo vệ tính ổn định và xây dựng bộ kiểm thử toàn diện là yếu tố quyết định chất lượng phần mềm.
[VIẾT NỘI DUNG CỦA SINH VIÊN TẠI ĐÂY]

5.3. Hướng phát triển phù hợp

Hướng phát triển tiếp theo của đề tài gồm: (1) Nghiên cứu cài đặt Sắp xếp chèn nhị phân (Binary Insertion Sort): ứng dụng tìm kiếm nhị phân để giảm số phép so sánh khóa xuống O(log n); (2) Phát triển thuật toán Shellsort: chia mảng thành các bước nhảy h-gap để đưa dữ liệu về trạng thái gần tăng trước khi chèn trực tiếp, giúp cải thiện độ phức tạp xuống O(n log^2 n); (3) Tích hợp thuật toán chèn như một pha hoàn tất trong các thuật toán lai hiện đại như Timsort hoặc Introsort.
[VIẾT NỘI DUNG CỦA SINH VIÊN TẠI ĐÂY]

Bảng 5.1 Mức hoàn thành và bằng chứng theo yêu cầu

Mục tiêu	Mức hoàn thành	Bằng chứng	Giới hạn
Cài đặt thuật toán & hàm kiểm tra	100%	Mã nguồn src/DeTai.jl chạy đúng mọi trường hợp	Giới hạn mảng n <= 5000 theo phiếu
Bộ kiểm thử tự động >= 12 ca	100%	test/runtests.jl đạt 42/42 asserts (Pass 100%)	Không có

Mục tiêu	Mức hoàn thành	Bằng chứng	Giới hạn
Lần vết tay & khảo sát thực nghiệm	100%	Bảng lần vết tay mảng [4,1,3,2], khảo sát 4 quy mô	Không có
Xuất bản GitHub & minh chứng ảnh	100%	11 ảnh gốc docs/anh_ket_qua/, SHA C & clone sạch	Không có

TÀI LIỆU THAM KHẢO

1. Hồ Nhựt Minh, Giáo trình Cấu trúc dữ liệu và Giải thuật, Mã môn học INT1306, Học viện Công nghệ Bưu chính Viễn thông, Chương 2: Các thuật toán sắp xếp và tìm kiếm, Mục 2.2.3: Sắp xếp chèn trực tiếp, Bảng 2.3, Bài tập 16, 17, 19, trang 37–39, 61, 2026.

2. Thomas H. Cormen, Charles E. Leiserson, Ronald L. Rivest, Clifford Stein, Introduction to Algorithms (3rd Edition), MIT Press, Chapter 2: Getting Started (Insertion Sort, Analysis, Loop Invariants), pp. 16–29, 2009.

3. Julia Language Documentation, Arrays and Sorting Algorithms: InsertionSort and Stable Sorting, Official Documentation: https://docs.julialang.org/en/v1/base/sort/, 2026.

PHỤ LỤC

PHỤ LỤC A. Tổ chức mã và hướng dẫn chạy

Mã nguồn đề tài được cấu trúc chuẩn hóa: N24DCDT018_DT012_DoHongDuong/

```
├── .gitignore                  # Cấu hình bỏ qua tệp tạm
├── README.md                   # Tài liệu thuyết minh và hướng dẫn chạy
├── demo.jl                     # Kịch bản thực nghiệm trực quan
├── chay_anh.jl                 # Kịch bản hỗ trợ chụp ảnh A01-A08
├── benchmark.jl                # Script khảo sát hiệu năng 4 quy mô
├── src/DeTai.jl                # Module thuật toán chính và validator
├── test/runtests.jl            # Bộ kiểm thử 12 ca độc lập
└── docs/anh_ket_qua/           # Chứa 11 ảnh minh chứng thực nghiệm
```

Lệnh thực thi chính trên PowerShell:

- Chạy demo trực quan: julia --startup-file=no demo.jl
- Chạy chế độ nhập từ bàn phím: julia --startup-file=no demo.jl -i
- Chạy bộ kiểm thử tự động: julia --startup-file=no test/runtests.jl

```
[CAY THU MUC MA NGUON]
[LENH CHAY VI DU 1]
[LENH CHAY VI DU 2]
[LENH NHAP DU LIEU MOI]
[LENH CHAY BO KIEM THU]
```

PHỤ LỤC B. Danh mục bằng chứng

Bảng 31 và thư mục docs/anh_ket_qua/ liệt kê đầy đủ 11 ảnh minh chứng thực nghiệm từ A01 đến A11, tương ứng với Hình 4.1 đến Hình 4.11 trong báo cáo, được chụp trực tiếp từ môi trường chạy thật tại commit SHA C: 72b4a97b2b7c2a52a39d9eed1909bd4eceb451ac.

Mã ảnh	Ca hoặc đối tượng	Tệp ảnh log và SHA
A01	Ví dụ 1 trên phiếu [4, 1, 3, 2]	docs/anh_ket_qua/01_vi_du_1.png | SHA: 72b4a97
A02	Ví dụ 2 trên phiếu [1, 2, 2, 3]	docs/anh_ket_qua/02_vi_du_2.png | SHA: 72b4a97
A03	Ca tự thiết kế 1 [1, 3, 2, 4, 5]	docs/anh_ket_qua/03_tu_thiet_ke_1.png | SHA: 72b4a97
A04	Ca tự thiết kế 2 [2, 1, 4, 3, 6, 5]	docs/anh_ket_qua/04_tu_thiet_ke_2.png | SHA: 72b4a97
A05	Ca biên 1: Mảng rỗng Int[]	docs/anh_ket_qua/05_bien_1.png | SHA: 72b4a97
A06	Ca biên 2: Mảng một phần tử [42]	docs/anh_ket_qua/06_bien_2.png | SHA: 72b4a97
A07	Ca biên 3: Mảng trùng nhau [5, 5, 5, 5]	docs/anh_ket_qua/07_bien_3.png | SHA: 72b4a97
A08	Input sai bị Validator từ chối	docs/anh_ket_qua/08_input_khong_hop_le.png | SHA: 72b4a97
A09	Tổng kết toàn bộ 12 ca kiểm thử (42/42 Pass)	docs/anh_ket_qua/09_tong_ket_kiem_thu.png | SHA: 72b4a97
A10	Clone sang thư mục mới và chạy lại sạch	docs/anh_ket_qua/10_clone_sach_chay_lai.png | SHA: 72b4a97
A11	Trang GitHub repository và commit của sinh viên	docs/anh_ket_qua/11_github_repo_commit.png | SHA: 72b4a97

PHỤ LỤC C. Tự kiểm trước khi nộp

Bảng tự kiểm tra đối chiếu danh mục sản phẩm trước khi nộp bài:
[x] Thông tin sinh viên và mã đề tài khớp chính xác với bảng phân công (N24DCDT018 - DT012).
[x] Mã nguồn tự cài đặt đầy đủ theo yêu cầu phiếu giao, không dùng thư viện ngoài.
[x] Bộ kiểm thử gồm 12 ca độc lập đạt 42/42 kiểm định (100% PASS).
[x] Đầy đủ 11 ảnh minh chứng thực tế rõ nét trong báo cáo và thư mục docs/anh_ket_qua/.
[x] Phân tích lý thuyết, bảng lần vết và khảo sát hiệu năng trên 4 quy mô đầy đủ.
[x] Kho lưu trữ GitHub công khai, permalink và SHA C/S chính xác, bản clone sạch chạy thành công.
[x] Điền đầy đủ hai trang bìa, xóa bỏ toàn bộ hướng dẫn thừa, cập nhật mục lục và xuất PDF chuẩn.

☑ MSSV và đề tài đúng bảng phân công: N24DCDT018 — DT012.

☑ Phần tự cài, tham khảo và công cụ hỗ trợ đã khai báo đầy đủ và trung thực.

☑ Đủ 12 ca kiểm thử độc lập (TC01–TC12), đủ 2 ví dụ và 5 ca phiếu, 11 ảnh thực nghiệm thật.

☑ Có bảng truy vết chi tiết, chứng minh tính đúng, phân tích O và khảo sát 4 quy mô hợp lệ.

☑ URL GitHub công khai, SHA C (72b4a97), permalink và bản clone sạch đã chạy lại thành công.

☑ Đã thay thế toàn bộ hướng dẫn/ngoặc vuông, cập nhật mục lục và xuất PDF hoàn chỉnh.

PHỤ LỤC D HƯỚNG DẪN NỘP CLASSROOM

Hồ sơ nộp bài đồ án học phần INT13185 gồm đúng 4 tệp với tiền tố thống nhất N24DCDT018_DT012_DoHongDuong:
1. N24DCDT018_DT012_DoHongDuong_BaoCao.docx (Báo cáo Word hoàn chỉnh)
2. N24DCDT018_DT012_DoHongDuong_BaoCao.pdf (Báo cáo PDF xuất từ Word)
3. N24DCDT018_DT012_DoHongDuong_MaNguon.zip (Gói mã nguồn, test và ảnh sạch)
4. N24DCDT018_DT012_DoHongDuong_ThongTinNop.txt (Tệp thông tin nộp điền đầy đủ)

Bảng PL.1 Bốn tệp cần nộp

Tệp	Tên theo mẫu	Nội dung cần có
Báo cáo Word	N24DCDT018_DT012_DoHongDuong_BaoCao.docx	Báo cáo chi tiết 5 chương, đầy đủ bảng biểu và 11 ảnh
Báo cáo PDF	N24DCDT018_DT012_DoHongDuong_BaoCao.pdf	Xuất từ bản Word cuối cùng, chuẩn A4
Mã nguồn ZIP	N24DCDT018_DT012_DoHongDuong_MaNguon.zip	src/, test/, demo.jl, benchmark.jl, README.md, docs/
Thông tin nộp TXT	N24DCDT018_DT012_DoHongDuong_ThongTinNop.txt	Điền theo mẫu với SHA C, SHA S, permalink và kết quả

Không nộp nguyên gói mẫu của giảng viên như ZIP cá nhân. Giải nén ZIP của mình ở thư mục mới phải chạy được theo README. Không gói .git, bộ cài, môi trường ảo hoặc cache vào ZIP bài nộp.

C là commit mã/test/input đã chạy và chụp; S là commit chốt bài sau khi thêm ảnh/báo cáo. Clone S sang thư mục mới rồi chạy lại; nếu mã đổi sau C thì cập nhật test/ảnh chịu ảnh hưởng. SHA S ghi trong TXT nộp rời, không phải commit SHA của chính S vào S.

Các thông tin để chép vào TXT

URL GitHub thật: https://github.com/ResitaRias/DT012_SapXepChen_N24DCDT018

SHA C: 72b4a97b2b7c2a52a39d9eed1909bd4eceb451ac | SHA S: 387901a88441ca9fe3bca63fc706c7ba33bbee91

Permalink thuật toán: https://github.com/ResitaRias/DT012_SapXepChen_N24DCDT018/blob/72b4a97b2b7c2a52a39d9eed1909bd4eceb451ac/src/DeTai.jl
Permalink kiểm thử: https://github.com/ResitaRias/DT012_SapXepChen_N24DCDT018/blob/72b4a97b2b7c2a52a39d9eed1909bd4eceb451ac/test/runtests.jl

Môi trường và phiên bản: Windows 11 64-bit, PowerShell 5.1, Julia 1.12.6, Git 2.55.0

Lệnh chạy hai ví dụ: julia --startup-file=no demo.jl | Lệnh bộ test: julia --startup-file=no test/runtests.jl

Kết quả clone S: Kiểm tra đạt 100% | Số ca kiểm độc lập: 12 ca | Số phép kiểm đạt: 42/42 (Pass 100%)

Vào đúng bài tập trên Classroom, đính kèm đủ bốn tệp, mở kiểm tra lại rồi bấm Nộp bài. Kiểm tra trạng thái đã nộp; ảnh rời đã nằm trong báo cáo và ZIP nên không cần gắn lại 11 tệp ảnh riêng.